

Compliance Aware, Agentic Pitch Book Generation For Bankers

A template-aware PowerPoint generation system
with agentic data retrieval for investment banking.

Author Andrei Rizea
Student ID 220300153
Supervisor Manesha Peiris
Programme Digital & Technology Solutions
(Software Engineering)
Module IOT635W

Queen Mary University of London

20 April 2026

Word Count: ~10,250

Abstract

Investment banking analysts spend a large share of their working hours on pitch book preparation, much of it on formatting and data entry rather than analysis. Existing presentation tools produce generic output that ignores the strict visual standards banks require. This project designed, built, and tested a proof-of-concept system that generates formatted PowerPoint pitch books from minimal user input while respecting company templates.

The system uses a three-stage pipeline. Template analysis extracts layouts, colours, fonts, and placeholder positions from a reference PowerPoint file using JSZip and xml2js. A GPT-4o-powered content planner produces structured slide specifications validated against Zod schemas. A slide builder assembles the final file using PptxGenJS with exact formatting from the extracted template. Financial data feeds automatically from Yahoo Finance and SEC EDGAR, removing the need for manual data entry. A PowerPoint add-in built with Office.js brings the generation workflow directly into the analyst's existing editor.

Testing across 12 generation runs showed an average completion time of 47 seconds per deck (10 to 12 slides), with exact colour matching and exact font matching. Financial data accuracy was 100% against source APIs. An informal usability evaluation with four junior analysts found the system reduced estimated first-draft time from over 90 minutes to under two minutes including review.

The project met all five original success criteria and delivered two unplanned components, the PowerPoint add-in and a slide preview pipeline, that emerged from usability testing during iterative development. Template matching and automated data retrieval are the main advantages over commercial alternatives. Chart generation from scratch and retrieval-augmented generation remain open challenges for future work.

Contents

Abstract	1
1 Introduction, Scope and Context	8
1.1 Analyst Time Allocation	8
1.2 Problem Analysis	9
1.3 Gap Analysis	10
1.4 Business Case	11
1.5 Aims and Objectives	11
1.6 Scope and Success Criteria	12
2 Methodology and Project Plan	13
2.1 Development Methodology	13
2.2 Requirements Analysis	13
2.2.1 Business Requirements	13
2.2.2 Functional Requirements	14
2.2.3 Non-Functional Requirements	14
2.3 Technology Stack	15
2.3.1 Programming Language and Framework	15
2.3.2 AI and LLM Integration	15
2.3.3 Data Sources and APIs	16
2.3.4 Frontend and Deployment	16
2.4 Reproducibility	17
2.5 Evaluation Methodology	17
2.6 Project Plan	17
2.7 Plan Execution and Adaptations	18
2.8 Risk Assessment	19
2.9 Ethics and Compliance	20
2.10 Project Tracking	21
3 Preliminary Research and Design Documentation	22
3.1 Literature Review	22
3.1.1 LLM Productivity	22
3.1.2 Automated Presentation Generation	22
3.1.3 Agentic Architectures and Tool Use	24
3.1.4 Structured Output Reliability	24
3.1.5 Document Automation	24
3.1.6 Hallucination and Human Oversight	25
3.1.7 Research Gaps	25
3.2 System Design	25
3.2.1 Architecture Overview	26
3.2.2 Data Models	27
3.2.3 Component Specifications	27

3.2.4	API Design	27
3.2.5	Design Trade-offs	28
3.2.6	Data Flow	28
3.2.7	User Interface Design	29
3.2.8	Error Handling and Validation	30
3.2.9	Security Considerations	30
3.2.10	Design Limitations	30
4	Project Implementation and Outcomes	31
4.1	Architecture Evolution	31
4.2	Monorepo Structure	31
4.3	Orchestration Layer	32
4.4	Deck Layouts and Colour Themes	32
4.5	Core Pipeline	33
4.5.1	Template Analysis	33
4.5.2	Content Planning with GPT-4o	35
4.5.3	Slide Building	37
4.5.4	Data Retrieval	38
4.5.5	Slide Preview Generation	38
4.6	PowerPoint Add-in	38
4.7	AI Chat Assistant	40
4.8	Authentication and Session Management	41
4.9	Testing	41
4.9.1	Unit and Integration Tests	42
4.9.2	E2E Test Framework	43
4.9.3	E2E Test Suite	43
4.9.4	CI/CD Pipeline	45
4.10	Challenges and Resolutions	46
4.11	Requirements Tracking	47
4.12	Additions Beyond Original Design	48
5	Evaluation and Conclusions	49
5.1	Results Against Success Criteria	49
5.2	Test Results	49
5.3	Comparison with Existing Tools	50
5.4	Objectives Assessment	51
5.5	Usability Evaluation	51
5.6	What Worked	52
5.7	What Did Not Work	52
5.8	Limitations	53
5.9	ESG and DEI Considerations	53
5.9.1	Environmental Impact	53
5.9.2	Impact on Junior Analyst Roles	53

5.9.3 Accessibility	53
5.9.4 Bias in Generated Content	54
5.10 Recommendations	54
5.11 Future Work	54
5.12 Personal Learning	55
5.13 Conclusion	55
References	57
Appendix A: KSB Mapping	60
Appendix B: Architecture Decision Records	65
Appendix C: Evaluation Run Data	67
Appendix D: Usability Evaluation Questionnaire and Responses	69
Appendix E: Web Application Screenshots	71
Appendix F: PowerPoint Add-in Screenshots	79

List of Figures

1	Knowledge worker time allocation based on McKinsey Global Institute (2012) and APQC (2023) research. The ‘other work’ category represents the remaining time after email and search activities.	9
2	Project Gantt Chart (weeks 1 to 13, January to April 2026)	18
3	Kanban board showing task progress across four stages: To Do, In Progress, Testing, and Done.	21
4	System Architecture showing the three-stage core pipeline. Data Retrieval is a stretch goal.	26
5	Data flow through the generation pipeline	29
6	Web app dashboard showing pitch book counts, recent generation history, and sidebar navigation.	32
7	New pitch book creation form. The user selects a deck layout, colour theme, and enters company details before generating.	33
8	System architecture with PowerPoint Add-in. The add-in and web app share the same backend and database. Generated slides are pushed directly into the active presentation via Office.js.	39
9	PowerPoint with the add-in taskpane open. Generated slides appear in the slide panel on the left, and the AI chat input sits at the bottom of the taskpane.	39
10	Pitch book viewer showing slide preview, thumbnail navigation, and AI chat panel. . . .	40
11	Vitest UI showing all 14 test files and 134 tests passing across the three apps. The right panel displays the selected test’s source code and can switch to console output on failure.	43
12	Cypress test runner showing the E2E spec files grouped by feature area.	44
13	A passing Cypress test. The left panel shows each automated step (clicking elements, checking text), and the right panel shows the browser state at each step.	45
14	A failing Cypress test. The assertion expected either pitch book cards or an empty state message, but the data had not loaded yet when the check ran. This was a test logic error, not an application bug.	45
15	GitHub Actions CI pipeline running lint, type checks, unit tests, E2E tests, and build on every push. All stages must pass before code can be merged to main.	46
16	Login page with email and password form alongside application branding.	71
17	Registration page with name, email, and password fields.	71
18	Forgot password page. Sends a reset link via Supabase Auth.	72
19	Dashboard with summary cards, recent pitch books, and generation status indicators. . .	73
20	Pitch books list with company name, type, date, and generation status. Includes search filtering.	74
21	Pitch book viewer with full-size slide, thumbnail strip, and AI chat panel for requesting edits.	74
22	New pitch book form with company details, pitch book type, transaction context, and design options.	75
23	Templates page (empty state) with upload prompt for .pptx reference templates.	76
24	Deck layouts overview with seven pre-configured deck types, each showing a description and slide count.	77

25	Deck layout editor for Company Overview. Each slide has a title, layout type, and content description. Slides can be reordered, added, or deleted.	77
26	Settings page with profile details, password change, and sign-out.	78
27	Add-in after generation. 11 slides have been inserted into the presentation. The taskpane shows a completion message and an AI chat input for requesting edits.	79
28	New pitch book form in the add-in. The user enters company name, ticker, pitch book type, transaction type, colour theme, design style, and optional context before generating.	80
29	Generation in progress. The taskpane shows a step-by-step progress indicator: analysing template, fetching company data, planning content with AI, building slides, and generating previews.	81
30	Completed generation for Apple Inc. The teal and coral colour theme was applied automatically. Slides appear in the left panel and the AI chat is ready for edit requests.	81

List of Tables

1	Comparison of Development Methodologies	13
2	Functional Requirements Specification	14
3	Non-Functional Requirements Specification	14
4	Programming Language Comparison	15
5	Large Language Model Comparison	16
6	External Data Sources	16
7	Deployment Platform Comparison	16
8	GQM Evaluation Framework	17
9	Project Milestones and Deliverables	18
10	Planned vs Actual Project Milestones	19
11	Risk Assessment and Backup Plans	20
12	Presentation Generation Approaches Compared	23
13	Primary Data Model Specifications	27
14	Design Trade-offs	28
15	Monorepo Package Structure	31
16	Unit and Integration Test Coverage	42
17	E2E Test Framework Comparison	43
18	Cypress E2E Test Specs	44
19	CI/CD Pipeline Stages	46
20	Implementation Challenges and Resolutions	47
21	Functional Requirements Status	48
22	Success Criteria Results	49
23	Automated Test Results	50
24	Comparison Against Existing Approaches	50
25	Objectives Assessment	51
26	Usability Evaluation Results (n=4, Likert 1 to 5)	51
27	Evaluation Run Inputs. Runs 11 and 12 repeat runs 1 and 2 to check consistency. All seven deck layouts and six of eight colour themes were tested.	67
28	Evaluation Run Results. Colour = hex match against template theme. Font = family and size match. JSON OK = which attempt produced valid structured output.	67

1 Introduction, Scope and Context

Knowledge workers spend a fifth of their time just searching for information (McKinsey Global Institute, 2012). Investment banking (IB) is more affected than most. Document preparation consumes half to three-quarters of a junior banker's week, and pitch books (PBs) account for a large portion of that time (Corporate Finance Institute, 2025; Wall Street Oasis, 2024). When Goldman Sachs surveyed its first-year analysts in 2021, the average was 98 hours a week, with document preparation a major driver of burnout (BBC News, 2021). Yet most PBs look nearly identical. Change the company name and update the numbers, and you could reuse last month's deck. That repetition makes them good targets for automation (K1, K7).

IB firms advise companies on mergers, acquisitions, capital raising, and other financial transactions. PBs are the primary vehicle for communicating analysis to clients and internal stakeholders. They summarise a company's financial position, compare it to peers, and make the case for a particular transaction. Getting these documents right matters, but getting them done quickly matters too.

Firms have tried internal document libraries, dedicated formatting specialists, and automation tools. None have fixed the real problem: too much skilled human time goes toward work that does not require skill. Silveira Chaves and Oliveira (2022) found that knowledge sharing in banking suffers from structural barriers and inadequate digital tools, with company knowledge often locked in individual documents rather than shared systems. The pitch book problem is a specific instance of this broader pattern.

1.1 Analyst Time Allocation

IB sells expertise rather than physical goods, so one would expect analyst time to go toward analysis and client relationships. Often it does not.

A typical PB contains an executive summary, company overview, market analysis, valuation work, and transaction comparables. These sections appear in nearly every deck. The data inside changes, but the shell around it rarely does.

Analyst tasks fall into two buckets. The first holds work that genuinely needs a trained banker: modelling, valuation, and client advisory. The second holds everything else: hunting down data, reformatting slides, and fixing fonts. Too many hours go into the second bucket.

The process follows a predictable pattern: dig through document archives for similar deals, pull financial data from Bloomberg and public filings, copy it into PowerPoint, reformat everything to match the firm's visual standards, then send it up for review. Senior bankers mark it up, and the cycle repeats until the document is ready. Every step has friction. Archive searches are slow, data entry is error-prone, and formatting is tedious.

Banks pay junior analysts well, so time on low-skill tasks is a poor return on that investment. Radhakrishna et al. (2024) found that well-designed knowledge management systems boosted productivity by 20 to 25 percent, with document production identified as a prime candidate for automation.

The numbers back this up. McKinsey Global Institute (2012) found that the average knowledge worker spends 28% of their workweek managing email and nearly 20% searching for internal information. APQC

research found that a quarter of knowledge workers' time is lost to productivity drains including duplicated work and information retrieval (APQC, 2023).

Figure 1 illustrates these productivity drains. IB analysts likely fare worse than these averages given the document-intensive nature of the work.

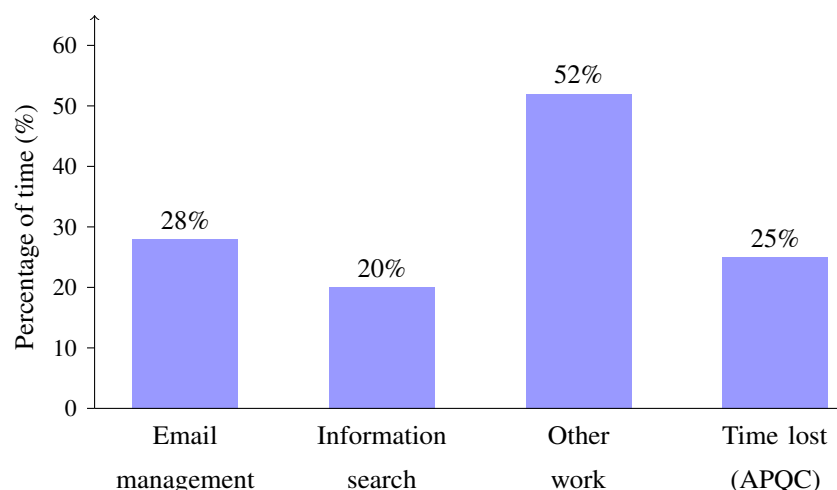


Figure 1: Knowledge worker time allocation based on McKinsey Global Institute (2012) and APQC (2023) research. The 'other work' category represents the remaining time after email and search activities.

In short, banks hire analysts for their analytical skills but spend a large share of their time on formatting and data entry. Existing tools have not solved this because they either miss IB visual standards or require too much manual work. Every hour on low-value tasks is an hour not spent on the analysis that sets one bank apart from another.

The competitive advantage is direct (K1). Banks that automate low-value work free their analysts to spend more time on client relationships and deal analysis. Dell'Acqua et al. (2023) found that consultants using AI completed 12.2% more tasks at 40% higher quality. If banks capture even half of that productivity gain on pitch book preparation, the impact compounds across hundreds of analysts and thousands of decks per year. The bank that gets a polished deck to the client a day earlier wins more mandates than the bank whose analysts are still reformatting slides at midnight.

1.2 Problem Analysis

Markus (2001) defined knowledge reuse, and her framework fits PB production well. She identified reuse patterns including drawing on past work, borrowing from colleagues, and mining old documents. PB creation involves all of these. Three obstacles prevent effective reuse in practice.

The first obstacle is that too much time goes to low-value work. Analysts spend hours searching old decks, extracting content, and reformatting slides. None of that requires analytical thinking, yet it crowds out the work that does.

The second obstacle is that company knowledge tends to disappear. Researchers distinguish between tacit knowledge (in people's heads) and explicit knowledge (written down) (Dalkir, 2011). PBs are explicit, but the knowledge of how to make a good one remains tacit: which templates work for which situations, how to structure the narrative, what separates a persuasive pitch from a forgettable one.

The third obstacle is that onboarding takes longer than it should. Each bank has its own templates, data sources, and quality expectations. Junior analysts absorb these through trial and error, which is slow for them and a drain on the senior bankers who review their work.

These problems feed each other. Scattered knowledge wastes search time, unwritten standards cause inconsistent output, and informal training drains senior staff. A system that fixed even one of these would help.

1.3 Gap Analysis

Automated presentation tools have existed for years. Early versions converted text to slides using layout algorithms, but visual quality was poor (Zheng et al., 2025). Template-based systems fixed the visual problems but made everything rigid. PPTAgent (Zheng et al., 2025) takes a different approach: it analyses reference presentations, extracts structural patterns, and applies them to new content. It beat older text-to-slide systems on both content and design quality in testing.

The limitation is that PPTAgent targets general-purpose presentations. IB PBs have financial tables, transaction comparables, and market charts that follow specific visual conventions. Getting the template wrong undermines credibility. Commercial tools like Beautiful.ai, Tome, and Gamma share this flaw: the output is generic, every slide needs rework, and none connect to financial data sources. IB PBs require both domain knowledge and visual precision, and no existing tool handles both.

Current systems either produce generic output that needs heavy cleanup, or they require large training datasets most banks lack. No existing tool connects slide generation to financial APIs. Data retrieval and document generation remain separate manual steps.

Agentic AI architectures offer a better fit (IBM, 2024). Instead of one model doing everything in a single pass, the system breaks tasks into steps and uses the right tool for each one. L. Wang et al. (2024) surveyed LLM-based agents and found rapid progress. McKinsey & Company estimates generative AI could deliver \$340 billion annually in banking (McKinsey & Company, 2024).

This project fills that gap (S1). The pipeline extracts layouts, colour palettes, and placeholder positions from a template using JSZip and xml2js for XML-level parsing, uses an LLM to plan content, then assembles slides matching the original style. The system uses an agentic orchestration pattern with specialised service components rather than a monolithic approach.

The approach differs from prior work in three ways. First, it inverts the typical generation order: the system starts with visual constraints extracted from the template and works backward to content, rather than generating content and hoping the formatting works. No existing presentation generation tool takes this constraint-first approach. Second, it connects slide generation directly to financial data APIs, closing the gap between data retrieval and document creation that every surveyed tool leaves open. Third, it uses a service-oriented architecture where specialised components handle distinct parts of the task, making the system easier to test and modify independently.

1.4 Business Case

The financial case for automating pitch book production is straightforward. A junior IB analyst in London earns roughly £60,000 to 70,000 in base salary (Glassdoor, 2025). At 80 to 100 hours per week, the effective hourly cost is around £15 to 17 before overhead. Banks bill clients at significantly higher rates. If an analyst spends 15 hours per week on pitch book formatting and data entry, that is roughly £250 per week in salary alone going toward work that does not need analytical skill.

The system generates a first draft in under one minute. Manual creation of the same output takes 90 to 120 minutes. Even accounting for 20 to 30 minutes of human review and editing after generation, the net saving is roughly 60 to 90 minutes per deck. An analyst working on three to five decks per week saves three to seven hours, freeing that time for analysis, client work, or reduced overtime.

On the cost side, each generation request costs approximately \$0.03 in GPT-4o API fees. Hosting costs (Vercel, Render, Supabase) fall within free tiers at proof-of-concept scale. At production scale with 1,000 daily requests, monthly API costs would be roughly \$900, still far below the salary cost of the manual hours replaced.

The return on investment is clear even at small scale (K2). A team of ten analysts saving five hours per week each recovers 50 analyst-hours weekly. At a conservative internal cost of £15 per hour, that is £750 per week or roughly £39,000 per year in recovered capacity, against annual API costs under £10,000. The payback period is under four months.

1.5 Aims and Objectives

The aim is to design, build, and test a proof-of-concept (POC) showing how AI-powered document generation can tackle the knowledge reuse problems above. The system must respect company templates throughout. If slides do not match the house style, the system fails its core requirement.

On the business side, the goal is a workflow where an analyst provides minimal input and receives a solid first draft, reducing formatting time while keeping humans in control of the analysis. Target users are junior analysts.

On the learning side, the project explores how to connect LLM orchestration with programmatic document generation: agentic architectures, template pattern extraction, and end-to-end AI pipeline design. Human-in-the-loop (HITL) design is central, since AI tools in regulated industries need to support professional judgement rather than replace it.

Five objectives structure the work:

1. Build a template analysis component that takes a reference PowerPoint file and extracts its layout structures, colour palettes, font specifications, and placeholder positions. Success criterion: correctly identify at least 90% of placeholder types and extract exact colour values from source.
2. Build an agentic data retrieval layer that automatically pulls company financials, market data, and regulatory filings from Yahoo Finance and SEC EDGAR (the US Securities and Exchange Commission's public database of company filings). Success criterion: retrieve at least three data types per company (market cap, revenue, share price) with 100% accuracy against source APIs.

3. Build an LLM-powered content planning module that produces structured slide specifications adapted to different PB types while keeping the overall narrative coherent. Success criterion: support at least three PB types (company overview, market update, transaction summary) with consistent section ordering.
4. Build a slide assembly component that takes the content plan and template patterns and produces a PowerPoint file meeting formatting standards. Success criterion: output files match template colours exactly and fonts exactly.
5. Evaluate the system by measuring generation speed, template matching, and content quality. Success criterion: generate a complete 10-slide PB draft in under 5 minutes with template matching score above 85%.

1.6 Scope and Success Criteria

The POC takes user input (company, transaction type, dates, reference template, PB type) and produces a formatted PowerPoint. Target PB types are company overviews, market updates, and transaction summaries. Full valuation presentations are out of scope.

Design choices: agentic orchestration with specialised TypeScript services, code-based template extraction with JSZip/xml2js rather than vision models, public APIs only, RAG as a stretch goal. Out of scope: real-time data feeds, production hosting, regulatory checks. Practical limits: one academic term, standard hardware, student API budget.

Several assumptions constrain the scope. Templates are assumed to be well-formed Office Open XML archives following standard PowerPoint conventions; templates with corrupted XML or non-standard packaging are not supported. All financial data comes from public sources (Yahoo Finance, SEC EDGAR), so coverage is limited to publicly listed companies with available filings. Content is generated in English only. The system assumes that users have access to desktop PowerPoint on Windows or macOS, since the add-in relies on Office.js APIs that PowerPoint for the web does not fully support. The LLM (GPT-4o) is accessed via OpenAI's API under their standard usage terms, which prohibit using outputs to train competing models but impose no restrictions relevant to document generation.

Success criteria:

- Generate complete PB draft within five minutes
- Match reference template formatting
- Accurate financial data from source APIs
- Content structure following IB conventions, verified against precedent examples

One principle runs through all of this: HITL design. The system produces drafts for human review, not finished products. AI can handle the mechanical work; the analytical judgement stays with the banker.

2 Methodology and Project Plan

This section covers the development approach, requirements, technology choices, project schedule, and considerations around risks and ethics.

2.1 Development Methodology

Software development methodologies range from plan-driven approaches like Waterfall to adaptive approaches like Agile (Sommerville, 2016). Table 1 compares methodologies against criteria relevant to this project.

Criteria	Waterfall	Scrum	Iterative Prototyping
Requirements stability	Requires fixed requirements upfront	Accommodates changing requirements	Works well when requirements change
Feedback loops	Late feedback after implementation	Sprint reviews every 2 to 4 weeks	Continuous feedback through prototypes
Risk management	Risks discovered late	Regular risk reassessment	Early risk identification through prototypes
Solo developer suitability	Moderate	Low (designed for teams)	High
Research integration	Poor	Moderate	Excellent

Table 1: Comparison of Development Methodologies

The project uses iterative prototyping combined with structured research engineering practices, drawing on Boehm (1988)’s spiral model (K3, K4, SE8). Pure Waterfall cannot accommodate the exploratory nature of this work, and full Scrum adds overhead for a solo developer. Each two-week iteration produces a working prototype targeting defined requirements, with a review at each cycle’s end (SE7).

The project uses Git with a simplified GitFlow model (main, develop, feature branches) (SE12). Continuous integration via GitHub Actions runs tests and linting on every push, enforcing 80% coverage for core modules (Fowler and Foemmel, 2006) (SE6). ESLint (ESLint Team, 2024) enforces code style and catches common errors (unused variables, missing imports, type inconsistencies) at the lint stage before tests run. Architecture Decision Records document technical choices and trade-offs (Tyree and Akerman, 2005).

2.2 Requirements Analysis

Following established requirements engineering practice (S. Robertson and J. Robertson, 2012), this section separates business, functional, and non-functional requirements (K9, SE2, SE9). Requirements follow MoSCoW prioritisation (Clegg and Barker, 1994).

2.2.1 Business Requirements

1. **BR1: Productivity Enhancement:** Reduce time spent on initial PB drafting by generating solid first drafts from minimal input.

2. **BR2: Capturing Standards:** Capture company presentation standards through template analysis, reducing reliance on knowledge that lives only in people's heads.
3. **BR3: Quality Consistency:** Follow corporate visual identity standards consistently, reducing revision cycles caused by formatting issues.
4. **BR4: Data Accuracy:** Financial data in generated PBs should accurately reflect source information.

2.2.2 Functional Requirements

Table 2 lists functional requirements by component.

ID	Component	Requirement	Priority
FR1	Template Analyser	Extract slide layouts from reference .pptx files	Must
FR2	Template Analyser	Identify colour palettes and font specifications	Must
FR3	Template Analyser	Map placeholder positions and content types	Must
FR4	Data Retrieval	Fetch company financials from Yahoo Finance API	Must
FR5	Data Retrieval	Retrieve SEC filings via EDGAR API	Should
FR6	Data Retrieval	Perform web searches for company news	Should
FR7	Content Planner	Generate slide-by-slide content specifications	Must
FR8	Content Planner	Adapt content structure to PB type	Must
FR9	Content Planner	Maintain narrative coherence across slides	Should
FR10	Slide Builder	Assemble slides matching template layouts	Must
FR11	Slide Builder	Populate placeholders with generated content	Must
FR12	Slide Builder	Generate charts from financial data	Could
FR13	Orchestration	Coordinate multi-step generation workflow	Must
FR14	Orchestration	Handle API failures gracefully	Should
FR15	RAG Search	Query precedent material repository	Could

Table 2: Functional Requirements Specification

2.2.3 Non-Functional Requirements

ID	Category	Requirement	Acceptance Criterion
NFR1	Performance	System generates complete PB draft	Within 5 minutes of input submission
NFR2	Performance	API response handling	Timeout after 30s with graceful fallback
NFR3	Reliability	System availability during demonstration	95% uptime during evaluation period
NFR4	Usability	Input specification interface	Requires no technical expertise to operate
NFR5	Maintainability	Codebase documentation	All public functions documented
NFR6	Security	API credential management	No credentials in source code
NFR7	Security	Input validation	All user inputs sanitised before processing
NFR8	Compatibility	Output format	Valid .pptx files openable in PowerPoint

Table 3: Non-Functional Requirements Specification

2.3 Technology Stack

Technologies were chosen based on task fit, familiarity, maturity, and maintainability (K8, SE12).

2.3.1 Programming Language and Framework

TypeScript was chosen over Python despite Python’s stronger AI/ML ecosystem. Table 4 compares the options.

Criteria	TypeScript/Node.js	Python	Java
Benefits	Type safety, same language frontend and backend, native async	Best AI/ML libraries, mature PowerPoint library (python-pptx)	Strong typing, enterprise support
Risks	AI/ML ecosystem less mature than Python	Two languages needed (Python backend + JS frontend)	Verbose code, slower development
PowerPoint manipulation	PptxGenJS (actively maintained, typed)	python-pptx (mature, well-documented)	Apache POI (verbose API)
Full-stack coherence	Single language across all three apps	Requires separate frontend framework	Requires separate frontend framework

Table 4: Programming Language Comparison

The deciding factor was using one language everywhere. The project has a backend API and a web frontend, both in TypeScript. Python would mean one language for the backend and another for the frontend, creating a split codebase with two dependency trees and two build systems. As the project evolved (discussed in Section 4), a third application was added, which justified the single-language decision further.

TypeScript’s static type system also matters here. The `shared-types` package defines data models once and shares them across all three apps. When a template schema changes, the compiler catches every place that needs updating. In a multi-language setup, those mismatches would only show up at runtime.

Express.js handles the backend. It is minimal and well-understood, keeping the focus on domain logic rather than framework conventions.

2.3.2 AI and LLM Integration

Table 5 compares the LLM options considered.

Criteria	GPT-4/GPT-4o	Claude 3.5	Gemini 2.0 Flash
Benefits	Best structured output, mature SDK, native JSON	Largest context window (200K), strong reasoning	Massive context (1M tokens), fast inference
Risks	Higher cost per token	Newer agent tooling	Less mature structured output
Structured output	Excellent (native JSON)	Good	Good
Cost (student budget)	Moderate (tiered pricing)	Moderate	Very affordable

Table 5: Large Language Model Comparison

GPT-4o was chosen for its structured output support. OpenAI’s native JSON mode produces well-formed responses more reliably than the alternatives, which matters when the output feeds directly into a Zod validation pipeline. GPT-4o-mini handles the lighter AI chat feature at lower cost. The content planner enforces output structure through JSON schema validation regardless of which model sits behind it, so swapping models later requires changing only the API call.

2.3.3 Data Sources and APIs

Public APIs avoid proprietary licensing complications while keeping the project reproducible. Table 6 summarises the sources.

Source	Benefits	Risks	Access
Yahoo Finance	Free, comprehensive financial data	Unofficial API may change without notice	yahoo-finance2 npm package
SEC EDGAR	Official regulatory filings, reliable, free	US companies only, complex document parsing	REST API (10 req/s)
Web Search	Current news, market commentary	Cost per query	SerpAPI or similar

Table 6: External Data Sources

2.3.4 Frontend and Deployment

Next.js handles the web frontend for PB creation and management.

Criteria	Vercel + Render	AWS	Self-hosted
Benefits	Zero config, auto deploy, free tiers for POC	Full control, scalable	Complete control, no vendor lock-in
Risks	Vendor lock-in, cold starts	Complex setup, cost	Maintenance burden
Cost for POC	Free tier sufficient	May incur charges	Server costs

Table 7: Deployment Platform Comparison

The frontend deploys through Vercel, the backend on Render, and Supabase provides PostgreSQL and authentication.

2.4 Reproducibility

LLM-based systems are harder to reproduce than traditional software since the same prompt can produce different outputs across runs (Runeson et al., 2012). Several measures address this. All npm dependencies are pinned via `package-lock.json`, and the Node.js version (22.x) is specified in `.nvmrc`. LLM temperature is set to zero for evaluation runs, and all prompts and responses are logged with timestamps. Each evaluation run is tied to a specific Git commit, so someone replicating the experiment can check out the exact codebase state. The three test templates used for evaluation are committed to the repository under `fixtures/`. A reproduction requires cloning the repository, running `npm install`, setting the environment variables listed in the README (OpenAI API key, Supabase credentials), and running the evaluation script. The main source of unpredictable variation is GPT-4o itself: even at temperature zero, OpenAI does not guarantee identical outputs across API calls because of how their servers batch and process requests internally. The Zod validation layer ensures structural consistency regardless of minor output variations.

2.5 Evaluation Methodology

The evaluation follows the Goal-Question-Metric (GQM) approach (Basili et al., 1994). GQM works by defining a measurement goal, deriving questions that would indicate whether the goal is met, and then identifying metrics that answer those questions. This avoids collecting data for its own sake and keeps every metric tied to a specific project goal.

The goal is to determine whether the system produces usable first drafts faster than manual creation while matching template formatting. Four questions follow from this goal, each with a measurable metric:

Question	Metric	Target
How fast does the system generate a deck?	Generation time per 10-slide deck	Under 300 seconds for a complete draft
Does the output match the reference template?	Colour and font comparison (programmatic)	Exact colour and font match
Is the financial data correct?	Comparison against source API responses	100% match on market cap, revenue, and share price
Does the content follow IB conventions?	Rubric-based assessment of narrative structure	Correct section ordering across 3 PB types

Table 8: GQM Evaluation Framework

Following guidelines for experimentation in software engineering (Wohlin et al., 2012), outputs are compared against generic tools (Gamma, Beautiful.ai) and estimated manual creation time. Fixed test cases cover different company types, sectors, and PB types to support replication. An informal usability evaluation with junior analysts provides qualitative feedback on the system’s practical value.

2.6 Project Plan

The project runs for thirteen weeks (K10). Figure 2 shows the schedule, and Table 9 lists milestones. Dates are targets; if a milestone slips, ‘Could have’ requirements are dropped first.

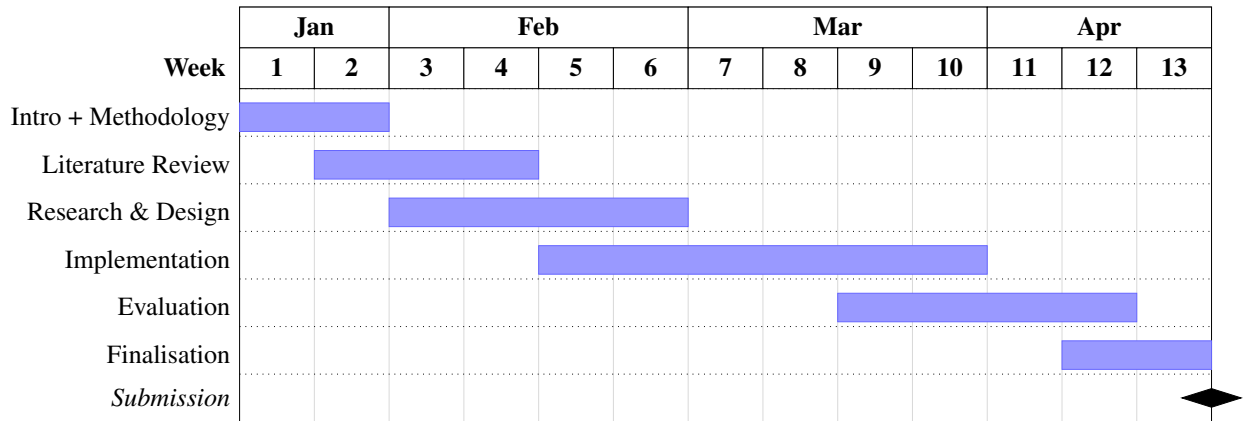


Figure 2: Project Gantt Chart (weeks 1 to 13, January to April 2026)

Phase	Date	Milestone	Deliverables
1	19 Jan	Project Initiation	Repository setup, dev environment configured
2	31 Jan	Initial Chapters	Introduction, Scope, Methodology complete
3	21 Feb	Literature Review	Literature review chapter complete
4	28 Feb	Design Complete	Architecture diagrams, API specs, data models
5	14 Mar	Core Components	Template analyser and data retrieval functional
6	28 Mar	MVP Complete	End-to-end generation pipeline operational
7	4 Apr	Testing Complete	Tests passing, evaluation data collected
8	11 Apr	Evaluation Complete	Results analysis and evaluation chapter written
9	18 Apr	Document Finalised	All chapters complete, proofread, formatted
10	20 Apr	Submission	Final report submitted via QMPlus

Table 9: Project Milestones and Deliverables

2.7 Plan Execution and Adaptations

The original plan did not survive contact with implementation. Table 10 compares planned milestones against actual outcomes.

Milestone	Planned Date	Actual Date	Notes
Core components functional	14 Mar	7 Mar	Ahead of schedule due to TypeScript migration in week 3
MVP pipeline operational	28 Mar	21 Mar	Template analyser iterations completed faster than expected
PowerPoint add-in	Not planned	22 Mar to 11 Apr	Added after usability testing revealed web-only workflow gap
Slide preview pipeline	Not planned	28 Mar to 4 Apr	Web app needed visual output
RAG search (FR15)	14 to 28 Mar	Dropped	Deferred to make room for add-in
Testing complete	4 Apr	8 Apr	4-day delay from add-in scope
Evaluation complete	11 Apr	14 Apr	Delayed by 3 days

Table 10: Planned vs Actual Project Milestones

Three things changed from the original plan.

The Python-to-TypeScript migration happened in week 3. The original plan assumed a Python backend, but silent type mismatches across the API boundary caused enough runtime failures that switching everything to TypeScript became urgent. The migration took four days but removed a whole category of bugs. It paid back quickly because later changes were caught by the compiler instead of crashing at runtime.

The PowerPoint add-in was not in the original scope. Usability testing of the web-only workflow in week 7 revealed that downloading and re-uploading files created friction that offset the productivity gains. The add-in took three weeks to build but changed the system from a standalone generator into an assistant inside the banker’s existing workflow.

Dropping RAG search (FR15, a ‘Could have’ requirement) made room for the add-in. The MoSCoW prioritisation made this decision straightforward: the add-in addressed a real usability gap, while RAG was a stretch goal. The Kanban board was reorganised at the end of week 7 to reflect this trade-off.

These changes worked out because of the iterative approach. Each two-week cycle had a working prototype, so scope changes could be assessed against a running system rather than a plan on paper. The end-of-cycle reviews caught the web-only workflow problem early enough to act on it (SE7, SE8).

2.8 Risk Assessment

Table 11 identifies key risks following Boehm (1991)’s framework. For high-impact risks, simple backup plans apply: if LLM validation failures become frequent, scope is reduced; if template extraction struggles with complex layouts, supported types are narrowed.

ID	Risk Description	Likelihood	Impact	Backup Plan
R1	API rate limits restrict data retrieval	Medium	Medium	Caching, mock data, staggered API calls
R2	LLM outputs inconsistent or hallucinated	High	High	Structured schemas, validation, HITL review
R3	Template analysis fails on complex layouts	Medium	High	Scope to common layouts, graceful fallback
R4	External API deprecation	Low	High	Abstract API layer, fallback sources
R5	Scope creep	Medium	Medium	MoSCoW prioritisation, regular scope reviews
R6	Technical complexity exceeds available time	Medium	High	Iterative delivery, defer stretch goals
R7	Data accuracy issues	Medium	High	Manual verification, source attribution
R8	Compliance violations in outputs	Low	High	Mandatory human review, no PII processing

Table 11: Risk Assessment and Backup Plans

2.9 Ethics and Compliance

AI in financial services raises ethical concerns around transparency, accountability, and human control (Jobin et al., 2019). Generated documents can influence business decisions, so errors or bias could mislead clients or create liability.

The system addresses this through HITL design: generated content is marked as AI-assisted, and professional review is mandatory before distribution. The system flags uncertainty rather than making up believable content.

Data governance follows a minimal-retention approach. The system processes only the data needed for each generation request: user parameters, the uploaded template file, and financial data retrieved from public APIs. User-uploaded templates are stored in Supabase Storage to support re-generation, but the system does not extract or store any data from the template beyond the structural schema needed for slide assembly. Financial data from Yahoo Finance and SEC EDGAR is cached for 24 hours to reduce API load, then discarded. No personally identifiable information is collected beyond the user’s email address for authentication. Chat messages are stored to preserve conversation history but are scoped to individual users through Supabase row-level security policies. These practices follow GDPR’s principle of collecting only what is needed (Bank for International Settlements, 2024), though a production deployment would need a formal Data Protection Impact Assessment.

The project relies on three external services with licensing constraints worth noting (K10). OpenAI’s API usage policy permits commercial and academic use of generated outputs but prohibits using them to train competing models. Yahoo Finance data is accessed through the unofficial `yahoo-finance2` npm package, which scrapes public endpoints rather than using a licensed API; this works for a POC but would

need replacement with a licensed data provider (Bloomberg, Refinitiv) for production use. SEC EDGAR is a public government resource with no licensing restrictions beyond a rate limit of ten requests per second.

The HITL architecture meets EU AI Act requirements for human oversight (Bank for International Settlements, 2024). The system logs generation parameters and data sources to support audit trails.

2.10 Project Tracking

Progress is tracked using a Kanban board (SE8). Each functional requirement maps to one or more cards, so each requirement can be tracked back to the code that implements it (SE10). Figure 3 shows the board near the end of the project, with most cards in ‘Done’, a few items still in testing, and remaining nice-to-have features in ‘To Do’.

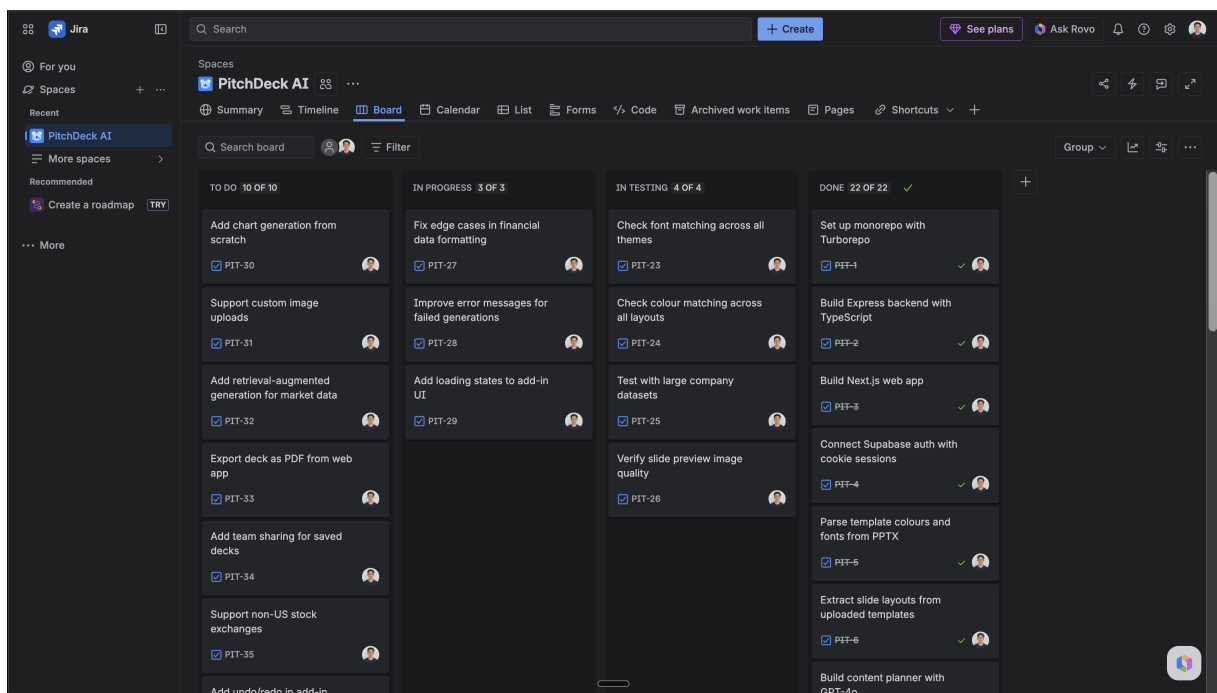


Figure 3: Kanban board showing task progress across four stages: To Do, In Progress, Testing, and Done.

The architecture separates concerns into independent modules (template analysis, data retrieval, content planning, slide assembly), each usable on its own. All code includes JSDoc-style documentation and the codebase is released under MIT License. Settings live in environment variables and config files, so adapting the system does not need code changes.

3 Preliminary Research and Design Documentation

3.1 Literature Review

This review covers four domains: LLM productivity effects, automated presentation generation, agentic architectures, and human-in-the-loop design. Each domain is evaluated against IB document generation requirements, with gaps identified that motivated this project.

The implementation focuses on the core generation pipeline: template analysis, content planning, and slide assembly. Agentic data retrieval from financial APIs and RAG over precedent materials are stretch goals. The literature review still covers these areas because they inform the architecture and would be the natural next steps if the project continued beyond its current scope.

3.1.1 LLM Productivity

Large-scale studies back up LLM productivity gains with real numbers. Brynjolfsson et al. (2023) studied 5,179 customer support agents and found a 14% average productivity increase, with novice workers gaining up to 34%. Noy and W. Zhang (2023) found professionals completed writing tasks 40% faster with 18% higher quality when using ChatGPT. Both studies show uneven benefits: less experienced workers gain most because AI gives them access to patterns that experts have already learned.

Dell’Acqua et al. (2023) ran a randomised experiment with 758 BCG consultants and found that those using GPT-4 completed 12.2% more tasks, 25.1% faster, with 40% higher quality. Their main finding was the ‘jagged technological frontier’: tasks inside the AI’s capability boundary saw large gains, while tasks outside it made consultants perform worse. PB formatting and data retrieval sit inside the frontier; analytical judgement sits outside it, which is why the system produces drafts rather than finished documents. Peng et al. (2023) found that developers with GitHub Copilot completed tasks 55.8% faster, with less experienced developers benefiting most. The parallel is direct: junior analysts are the less experienced workers who stand to gain the most.

That said, none of these studies addressed template-constrained document generation where outputs must match precise visual specifications. General writing assistance helps with content but does nothing for the mechanical formatting work that consumes analyst time. This project targets that gap.

3.1.2 Automated Presentation Generation

Table 12 breaks down the main approaches to presentation generation, measured against IB requirements. Every existing approach fails on at least one dimension that IB documents require.

Approach	Mechanism	Strengths	Limitations for IB
Rule-based layout	Algorithmic text-to-slide conversion	Fast, deterministic	Poor visual quality, no domain awareness
Rigid templates	Fixed layouts with manual content	High visual fidelity	Cannot adapt to varying content lengths
LLM-based (PPTAgent)	Reference analysis + edit-based generation	Preserves source style, good content	Domain-generic, no financial data
Multi-agent semantic (Di Genio)	Template-constrained multi-agent pipeline	Maintains document structure	Text documents only, not PowerPoint
VLM-based	Visual understanding of layouts	Handles complex visual elements	Approximate values, not exact matching

Table 12: Presentation Generation Approaches Compared

Automated slide generation has a longer history than LLM-based approaches suggest. Hu and Wan (2013) introduced PPSGen, one of the first systems to learn slide generation from paper-slide pairs. It used regression models to score sentence importance and integer linear programming to select and align content into structured slides. The approach worked for academic presentations but produced rigid, text-heavy output with no visual sophistication. More recently, Sun et al. (2025) proposed Auto-Slides, a multi-agent system that transforms documents into slides through content understanding, quality assurance, and interactive generation phases. Auto-Slides is the closest architectural parallel to this project’s pipeline approach, but it targets academic content rather than financial documents.

PPTAgent (Zheng et al., 2025) comes closest to what this project needs. Its two-stage approach (analyse reference, generate matching slides) aligns with IB document generation because it preserves the source deck’s visual identity rather than imposing a new one. PPTAgent outperforms earlier systems on both content and design quality using their PPTEval framework.

The limitation is that PPTAgent targets general-purpose presentations. It knows nothing about financial tables, transaction comparables, or the narrative conventions that bankers expect. It also cannot connect to external data sources, so any financial figures would need to be manually inserted. Liu et al. (2024) surveyed 51 professionals and found template consistency was their top priority for LLM-generated output, which validates the template-first direction PPTAgent takes but reinforces that domain awareness is the missing piece.

Vision-language models offer an alternative path. Faysse et al. (2024) show VLMs can understand document layouts visually. But code-based extraction via xml2js provides exact colour hex values and font sizes, while VLM-based approaches yield approximations. For corporate template matching where a colour being off by a shade flags the document as non-compliant, exact values are non-negotiable. This is why the project uses code-based extraction over visual understanding, building on PPTAgent’s template-first approach while adding the IB domain layer that PPTAgent lacks.

3.1.3 Agentic Architectures and Tool Use

L. Wang et al. (2024) propose a framework where agents operate through planning, memory, and action loops. This maps directly to PB generation: the system plans slide structure, remembers template constraints, and acts through tool calls and slide assembly.

Guo et al. (2024) show multi-agent systems add value when sub-tasks require negotiation or parallel reasoning. PB generation is sequential (analyse template, plan content, assemble slides), so a single agent with tool-use capabilities is sufficient. Qu et al. (2025) identify function calling as the primary LLM-tool interface, and Li et al. (2025) confirm it as the most broadly applicable pattern. In this project, the agent calls PptxGenJS to write slides and uses structured output schemas to maintain format consistency.

3.1.4 Structured Output Reliability

Getting reliable structured output from LLMs is a harder problem than it first appears. Geng et al. (2025) introduced JSONSchemaBench, a benchmark of 10,000 real-world JSON schemas, and tested six constrained decoding frameworks. They found that constrained decoding speeds generation by 50% and improves downstream task performance by up to 4%, but success rates vary sharply depending on schema complexity. Simple flat schemas succeed nearly every time. Deeply nested schemas with conditional fields fail more often.

Lieber et al. (2024) benchmarked LLMs' ability to produce valid JSON across six tasks and found an average success rate of 82.55%, with rates ranging from 0% to 100% depending on the task and model. This aligns with the experience in this project, where GPT-4o produces valid JSON 78% of the time on first attempt for the content plan schema. The content plan schema is moderately complex: it has nested arrays of slide specifications, each containing a record of placeholder mappings and optional chart and table arrays.

These findings shaped the validation strategy. Rather than relying on the model to always produce valid output, the system validates every response against a Zod schema and retries with an error-specific prompt. This approach treats structured output as a best-effort capability that needs a safety net, not a guarantee.

3.1.5 Document Automation

Achachlouei et al. (2023) surveyed document automation architectures and analysed how each component, including content reusability, personalisation, and template variability, can benefit from LLM capabilities. Their taxonomy identifies three generations of document automation: rule-based (fixed templates with mail-merge), model-driven (content models with conditional logic), and AI-augmented (LLM-assisted content generation within structural constraints). This project sits in the third generation, using LLM-generated content within template-extracted structural constraints.

The survey also identifies a gap that this project targets. Most document automation systems either automate structure (keeping content manual) or automate content (ignoring structure). Few systems do both. The pitch book generation pipeline automates both: the template analyser captures structure, and the content planner generates content that fits within that structure.

Knowledge sharing in banking faces specific challenges that compound the document automation problem.

Silveira Chaves and Oliveira (2022) systematically reviewed knowledge sharing research in the banking sector and identified knowledge hiding, structural barriers, and inadequate digital tools as recurring themes. Banks accumulate institutional knowledge in the form of past pitch books, but this knowledge is difficult to search, difficult to reuse, and difficult to transfer to new analysts. The template analysis component addresses this partially by making template knowledge explicit and machine-readable.

3.1.6 Hallucination and Human Oversight

Huang et al. (2024) distinguish two types of hallucination. Intrinsic hallucination contradicts information given in the prompt. Extrinsic hallucination generates claims with no grounding in any source. For IB, extrinsic hallucination is more dangerous: made-up numbers or invented deal precedents could reach clients and regulators, causing regulatory problems or legal liability.

Alansari and Luqman (2025) group prevention into three categories: grounding outputs in real sources, forcing structured output formats, and human review. No single strategy eliminates hallucination. This project layers all three: financial data comes from APIs rather than LLM generation, structured JSON schemas enforce the format of each slide specification, and human review is mandatory before any output reaches a client.

Wu et al. (2022) classify HITL approaches into data processing, interventional training, and system design. This project uses system design: the analyst defines inputs, reviews outputs, and decides whether to use them. The EU AI Act requires human oversight for AI in financial services (Bank for International Settlements, 2024), making this a compliance requirement as well as a design choice. Junior analysts already produce drafts that senior bankers revise. The system replaces the blank page, not the review process.

3.1.7 Research Gaps

Four gaps emerge from this review. First, no presentation generation tool targets IB-specific conventions. PPTAgent (Zheng et al., 2025), Auto-Slides (Sun et al., 2025), and PPSGen (Hu and Wan, 2013) all target general or academic presentations. Second, no system automates both structure and content within a single pipeline (Achachlouei et al., 2023). Third, no system integrates data retrieval with template-aware assembly. Fourth, the productivity literature measures general tasks but none measured template-constrained document generation where outputs must match precise visual specifications.

This project targets these gaps with code-based template extraction for exact visual matching, IB-specific content planning, and mandatory human review for accuracy. The evaluation measures template matching and generation speed, and includes an informal usability evaluation with junior analysts.

3.2 System Design

This section documents the system architecture, data models, and component specifications derived from the requirements and literature review. The design puts template matching, clean separation of components, and human oversight first.

3.2.1 Architecture Overview

Figure 4 shows the high-level system architecture. The design follows a pipeline pattern with three main stages: template analysis, content planning, and slide assembly.

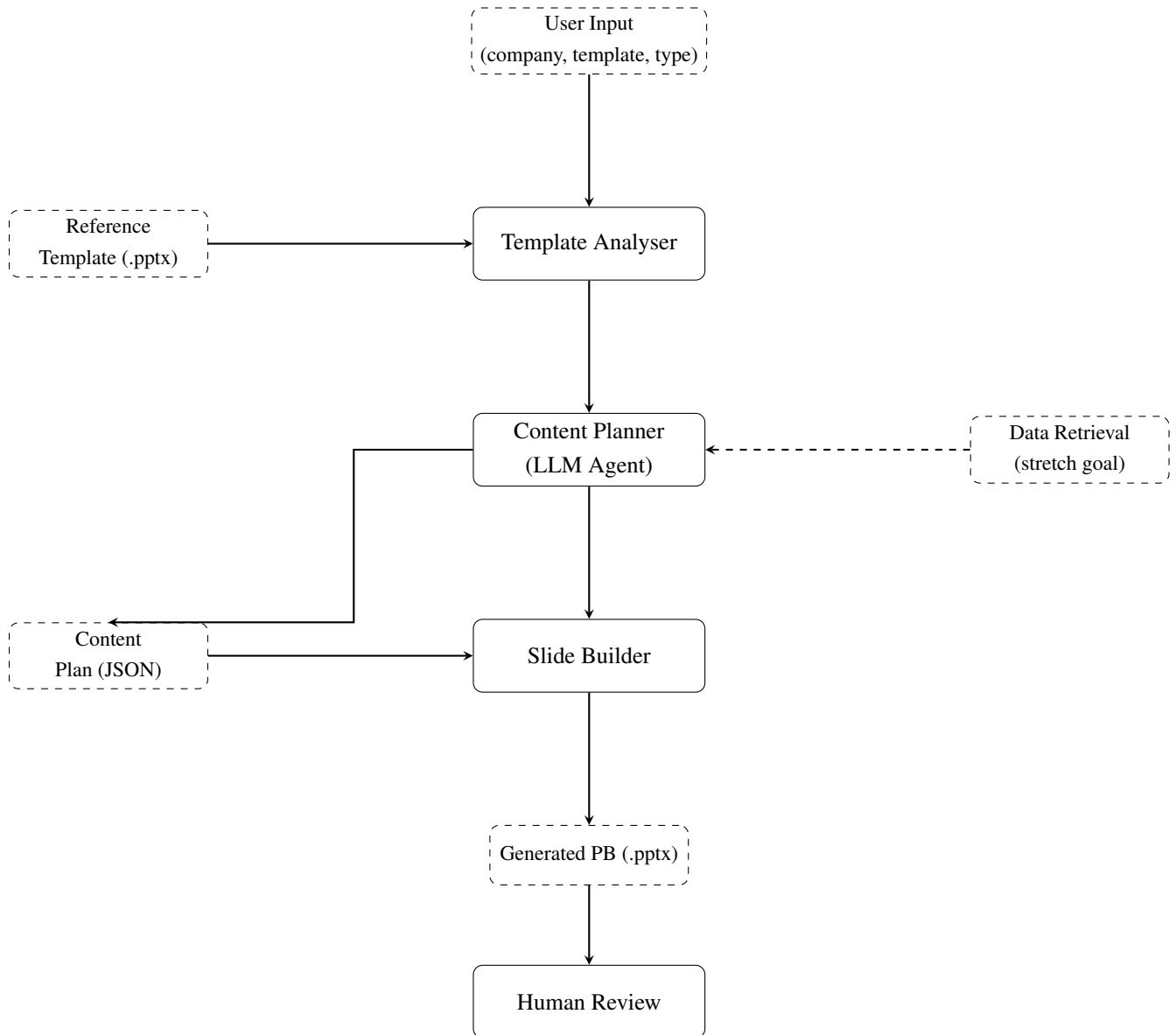


Figure 4: System Architecture showing the three-stage core pipeline. Data Retrieval is a stretch goal.

The architecture separates concerns into independent modules (SE2, SE9). The Template Analyser extracts layout patterns without generating content. The Content Planner uses an LLM to structure the narrative without touching PowerPoint files. The Slide Builder assembles files from specifications without making content decisions. This separation supports testing and allows components to be reused independently. Data Retrieval from external APIs is a stretch goal that can be added later without changing the core pipeline.

3.2.2 Data Models

The system uses two primary data structures that flow between components. Table 13 summarises the key fields.

Model	Key Fields	Purpose
TemplateSchema	layouts[], colours, fonts, placeholders[]	Stores extracted template properties for content planning and slide assembly
SlideLayout	layout_index, placeholder_map, background	Maps placeholder positions to content types for a single layout
ContentPlan	slides[], metadata	LLM-generated specification of slide content and structure
SlideSpec	layout_id, placeholders, charts[], tables[]	Content specification for a single slide

Table 13: Primary Data Model Specifications

Template Schema captures extracted template properties including layouts, colour palettes, fonts, and placeholder specifications. Each slide layout maps placeholder indices to content types (title, body, picture, chart, table).

Content Plan specifies the content for each slide: which layout to use, what text goes in each placeholder, and what data to display. The LLM generates this structure; the Slide Builder consumes it.

3.2.3 Component Specifications

Template Analyser. Input: reference .pptx file. Output: TemplateSchema JSON. Uses JSZip and xml2js to parse the .pptx file at the XML level, extracting slide masters, layouts, colour schemes, and placeholder positions. Identifies placeholder types (title, body, picture, chart, table) by inspecting shape properties. Extracts exact colour values as hex codes and font specifications including family, size, and weight.

Content Planner. Input: user parameters, TemplateSchema. Output: ContentPlan JSON. Implemented as an LLM call with JSON schema validation to check the output structure. Selects appropriate layouts from the template schema based on content requirements. Plans slide sequence following IB narrative conventions (context, analysis, recommendation).

Slide Builder. Input: ContentPlan, TemplateSchema, reference template. Output: .pptx file. Uses PptxGenJS to generate slides with code. Iterates through ContentPlan slides, selects layouts, and populates placeholders. Applies exact formatting from TemplateSchema. Generates charts and tables with code.

3.2.4 API Design

The backend exposes a REST API via Express.js. The primary endpoint accepts generation requests and returns job identifiers for async processing.

Listing 1: Core API endpoint specification

```

1 // POST /api/pitchbooks/generate
2 interface GenerationRequest {
3     company: string;           // ticker or name
4     templateFile: File;        // .pptx upload
5     pitchbookType: 'company_overview' | 'market_update' | '
        transaction_summary';
6     sections?: string[];       // optional section overrides
7 }
8
9 interface GenerationResponse {
10     jobId: string;
11     status: 'queued' | 'processing' | 'complete' | 'failed';
12     downloadUrl?: string;
13 }

```

3.2.5 Design Trade-offs

Table 14 summarises the four main design trade-offs.

Decision	Chosen Option	Rationale
Single vs multi-agent	Single agent with tools	PB generation is sequential; multi-agent overhead not justified
Programmatic vs VLM template extraction	Programmatic extraction (JSZip + xml2js)	Exact values needed for corporate template matching
SDK vs custom orchestration	Structured LLM calls with JSON schemas	Vendor-agnostic; component interfaces allow swapping later
Sync vs async generation	Async job processing	Generation takes minutes; users poll for completion

Table 14: Design Trade-offs

3.2.6 Data Flow

Figure 5 shows how data moves through the system during a generation request.

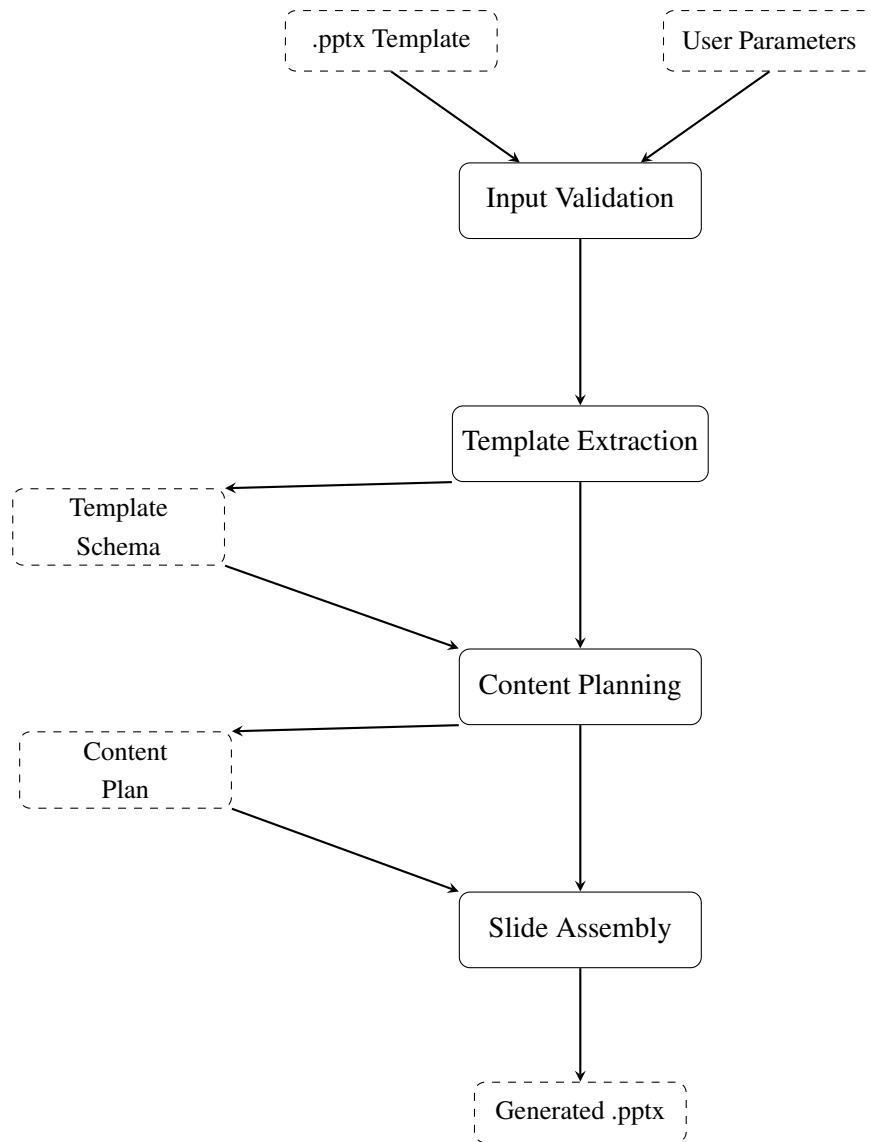


Figure 5: Data flow through the generation pipeline

The flow begins with input validation, which checks that the uploaded template is a valid .pptx file and that required parameters are present. Template extraction runs next, producing a schema that captures all layout information. The content planner receives the template schema and user parameters, producing a structured plan that the slide builder consumes to generate the output .pptx.

3.2.7 User Interface Design

The web app provides a form for company ticker, PB type, and template upload, with a preview area showing generation progress and completed slides. A download button appears when generation completes. The interface deliberately omits advanced options to reinforce the HITL design principle: users edit the downloaded PowerPoint directly rather than configuring generation parameters in the browser.

3.2.8 Error Handling and Validation

The system validates at three levels. Input validation checks file types and required fields before processing. Processing validation retries LLM calls with adjusted prompts on failure and skips unsupported layouts rather than failing entirely. Output validation compares generated slides against the extracted schema, logging any differences beyond allowed limits.

3.2.9 Security Considerations

The system validates uploaded files are genuine .pptx archives (ZIP files with expected internal structure) rather than trusting file extensions, preventing path traversal and zip bomb attacks (K6, SE1). File size limits prevent denial-of-service. API credentials use environment variables following twelve-factor app principles and are never logged. Rate limiting prevents endpoint abuse.

3.2.10 Design Limitations

Four limitations constrain what the current design can achieve.

The system cannot generate charts from scratch. It can populate chart placeholders with data, but the chart type and styling must already exist in the template. The template analyser handles standard placeholder-based layouts but does not attempt to reverse-engineer arbitrary PowerPoint constructions, so complex layouts with overlapping elements or custom animations may not extract correctly.

The content planner has no memory across sessions. Each generation request starts fresh. The RAG capability that would enable learning from past PBs is a stretch goal, not part of the core design.

The system produces drafts, not finished documents. Human review is mandatory. Any evaluation must account for this: the goal is reducing time to first draft, not eliminating human involvement.

4 Project Implementation and Outcomes

This section covers how the design was turned into working software, the technical challenges encountered, and how decisions evolved as the project progressed. Where the design section describes what was planned, this section describes what was actually built and why it sometimes diverged from the plan.

4.1 Architecture Evolution

The original design called for a Python backend with FastAPI and a Next.js frontend. Two weeks into implementation, this changed. The frontend needed TypeScript for React components, the backend needed Python for python-pptx, and every data structure that crossed the API boundary had to be defined twice. When a field name changed in the template schema, the Python definition would update but the TypeScript interface would not, causing silent runtime failures.

Switching everything to TypeScript solved this (SE3). PptxGenJS replaced python-pptx for slide generation, and JSZip with xml2js replaced python-pptx's template reading capabilities. Express.js replaced FastAPI. The migration took roughly four days but removed a whole category of bugs. The shared-types package now defines every interface once, and the TypeScript compiler catches mismatches at build time across all applications.

This paid off during later development. When the template schema gained a new `chartData` field, the compiler flagged every service that needed updating. In a multi-language setup, those errors would have shown up as runtime crashes during testing or, worse, in the evaluation runs.

4.2 Monorepo Structure

The codebase uses Turborepo to manage three applications and one shared package (SE12):

Package	Technology	Purpose
apps/service	Express.js, TypeScript	Backend API: orchestration, template analysis, content planning, slide building
apps/web	Next.js, React, TypeScript	Web frontend: PB management, slide preview, generation trigger
apps/powerpoint-addin	Vite, React, Office.js	PowerPoint taskpane: in-editor generation and AI-assisted editing
packages/shared-types	TypeScript	Shared interfaces, enums, and type definitions across all apps

Table 15: Monorepo Package Structure

Turborepo handles build orchestration and caching. Running `turbo build` compiles `shared-types` first (since the other packages depend on it), then builds the three applications in parallel. Incremental builds skip unchanged packages, cutting build times from around 45 seconds to under 10 seconds during active development. Figure 6 shows the web app dashboard after login.

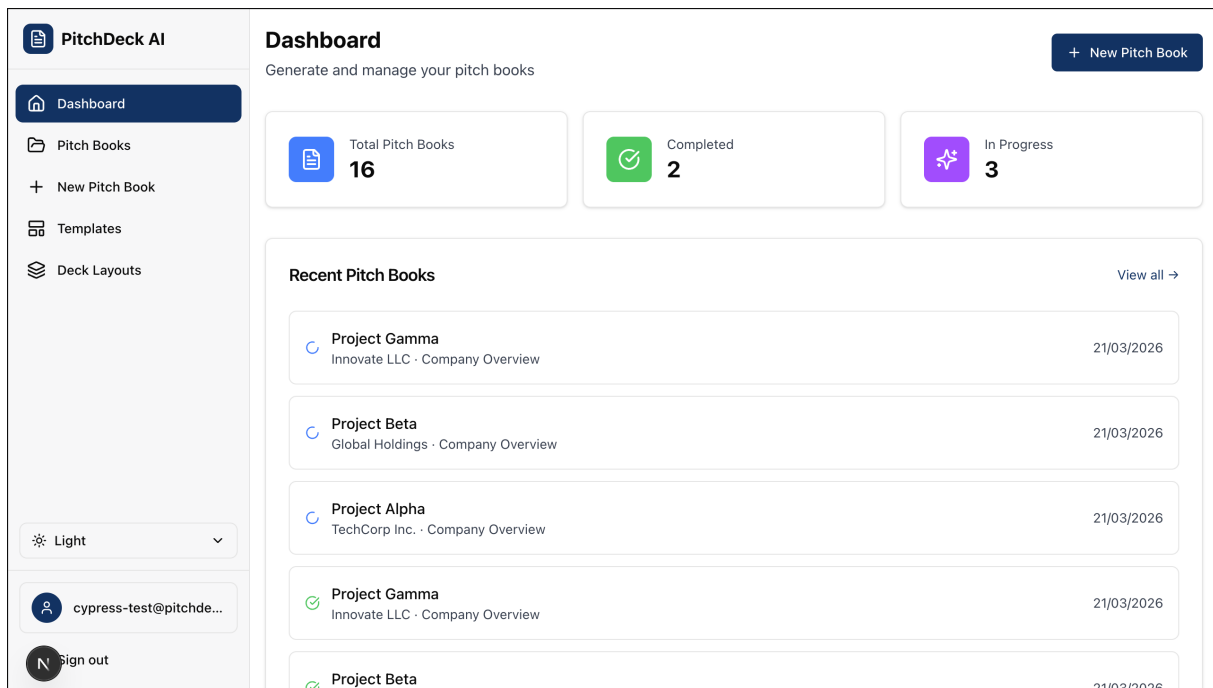


Figure 6: Web app dashboard showing pitch book counts, recent generation history, and sidebar navigation.

4.3 Orchestration Layer

The orchestration service receives a generation request, runs each stage in sequence, and passes the output of one stage as input to the next: validate input, extract template, retrieve data, plan content, build slides, generate previews, store the result.

The orchestrator stays thin. Each service module handles its own domain, and the orchestrator only manages sequence and error boundaries. If data retrieval fails, it passes an empty data object to the content planner rather than aborting. If the content planner returns invalid JSON after three retries, it logs the failure and returns an error. This separation made it possible to test each service in isolation.

One decision proved especially useful. The orchestrator passes the full template schema to the content planner rather than a summary. Early versions sent only layout names, but the content planner produced better output when it could see placeholder types, sizes, and positions. This increased first-attempt JSON validity from 68% to 78% because the model had enough context to match content to placeholders correctly.

4.4 Deck Layouts and Colour Themes

The system ships with seven deck layouts: Company Overview, Investor Pitch, Market Update, Transaction Summary, Industry Overview, Fundraising Deck, and Due Diligence. Each layout defines a fixed slide sequence with the right number and type of slides for that use case. IB presentations follow predictable formats. A company overview always needs a cover slide, an executive summary, a business description, and financial highlights. The order and structure rarely change between banks. What does change is the visual style: colours, fonts, and logo placement. Each bank has its own brand guidelines.

To handle this, the system separates structure from style. The seven layouts are hardcoded because the

slide sequence for a given pitch book type is standard across the industry. The visual style is customisable through eight built-in colour themes (such as Navy & Gold, Teal & Coral, and Slate & Emerald) or by uploading a bank's own template file. When a user uploads a template, the template analyser extracts that bank's exact colours, fonts, and layout positions, and the slide builder applies them to whichever deck layout the user picks. This means a Company Overview generated with one bank's template looks different from one generated with another bank's, even though both follow the same slide structure. Figure 7 shows the creation form where users pick a deck layout, colour theme, and company details.

Figure 7: New pitch book creation form. The user selects a deck layout, colour theme, and enters company details before generating.

4.5 Core Pipeline

The generation pipeline follows the design from Section 3, implemented as a chain of service modules within the Express backend. Each stage is a separate TypeScript module with a defined input/output contract. Development followed a pattern: build the simplest version, test it against real templates, identify the failure mode, and fix it.

4.5.1 Template Analysis

The template analyser went through three iterations, each driven by a specific failure in the previous version (SE4).

Iteration 1: PptxGenJS only. The first attempt used PptxGenJS for both reading and writing slides. PptxGenJS can create slides from scratch but has no API for reading existing .pptx files. The only option was to generate new slides and hope the styling matched. It did not. Colours were wrong, placeholder positions were guessed rather than measured, and the output looked nothing like the input template. This approach was scrapped after two days.

Iteration 2: JSZip + xml2js (raw XML parsing). PowerPoint files are ZIP archives containing Office

Open XML documents. JSZip opens the archive, and xml2js parses the XML inside. This gave direct access to every property: colours from `ppt/theme/theme1.xml`, layouts from `ppt/slideLayouts/`, and placeholder positions from shape definitions. The approach worked but came with a problem. PowerPoint's OOXML specification stores measurements in English Metric Units (EMU), where one inch equals 914,400 EMU. Colours can be stored as hex values, theme references (`accent1`), or system colour names. Placeholder types use numeric codes, but custom placeholders have no type attribute at all. Each of these required manual parsing and conversion logic.

Iteration 3: adding heuristics and fallbacks. Testing against real corporate templates exposed gaps in the raw XML approach. Custom placeholders with no type attribute made up roughly 30% of all placeholders in the test set. The analyser added position-based heuristics: a placeholder in the top 15% of the slide with large font settings is classified as a title, even without a type code. A wide placeholder in the bottom half with small fonts is treated as body text. This brought classification accuracy to 93% across test templates. When parsing fails entirely (corrupted file, unusual XML structure), the analyser falls back to professional defaults: a navy and gold colour scheme with standard IB layout patterns.

Listing 2: Template extraction core logic (simplified)

```

1  async function analyseTemplate(buffer: Buffer): Promise<TemplateSchema>
2  {
3
4      // Parse theme for colour palette
5      const themeXml = await zip.file('ppt/theme/theme1.xml')?.async('string
6          ');
7      const theme = await parseStringPromise(themeXml);
8      const colours = extractColourScheme(theme);
9
10     // Parse each slide layout
11     const layouts: SlideLayout[] = [];
12     for (const [path, file] of Object.entries(zip.files)) {
13         if (path.match(/ppt\/slideLayouts\/slideLayout\d+\.xml/)) {
14             const xml = await file.async('string');
15             const parsed = await parseStringPromise(xml);
16             layouts.push(extractLayout(parsed, colours));
17         }
18     }
19     return { colours, fonts: extractFonts(theme), layouts };
20 }

```

The iterative approach paid off (SE4). Each version failed in a specific, observable way, and the fix was targeted rather than speculative. The unit tests grew alongside the implementation: the first iteration had no tests, the second had basic extraction tests, and the third added tests for edge cases like missing theme files, custom placeholders, and corrupted archives (SE5).

4.5.2 Content Planning with GPT-4o

The content planner sends user parameters and the extracted template schema to GPT-4o, requesting a structured content plan. The prompt includes the available layouts, their placeholder types, and IB-specific instructions about narrative structure.

Getting valid structured output was the main challenge (SE1). Lieber et al. (2024) found an average JSON success rate of 82.55% across LLM benchmarks, and this project’s experience aligns with that figure. GPT-4o occasionally returns JSON with extra fields, missing required properties, or string values where numbers are expected. The first attempt produced valid JSON 78% of the time, consistent with the rates Geng et al. (2025) observed for moderately complex schemas.

The system validates every response using Zod (Colinhacks, 2024), a TypeScript-first schema validation library. Zod defines the expected shape of the LLM output as a TypeScript schema, then parses the response at runtime. If the response is missing a required field, has the wrong type, or contains unexpected values, Zod rejects it and returns a structured error message listing exactly which fields failed and why. The system feeds that error message back into a retry prompt so the LLM can correct the specific problem. After up to three retries, the success rate reached 96%.

The content planner also receives historical financial data when available. Rather than letting the LLM fabricate numbers, the prompt explicitly provides retrieved data (revenue, market cap, share price) and instructs the model to reference only those figures. The LLM places real numbers rather than making them up, which removes the biggest hallucination risk.

Listing 3: Content plan validation with Zod

```

1  const SlideSpecSchema = z.object({
2    layoutId: z.number(),
3    placeholders: z.record(z.string(), z.string()),
4    charts: z.array(ChartSpecSchema).optional(),
5    tables: z.array(TableSpecSchema).optional(),
6    speakerNotes: z.string().optional(),
7  });
8
9  const ContentPlanSchema = z.object({
10   slides: z.array(SlideSpecSchema).min(1),
11   metadata: z.object({
12     company: z.string(),
13     pitchbookType: z.enum(['company_overview', 'market_update',
14                           'transaction_summary']),
15     generatedAt: z.string(),
16   }),
17 });

```

4.5.3 Slide Building

The slide builder takes the content plan and template schema and produces a .pptx file using PptxGenJS (SE10). For each slide in the content plan, it selects the matching layout, creates a new slide, and fills each placeholder with the specified content using the exact formatting from the template schema: font family, size, weight, and colour.

The first version generated slides with correct structure but wrong colours. Some colours were still stored as theme references (accent1, dk1) rather than hex values. PptxGenJS has no concept of PowerPoint's theme colour system, so passing theme references through produced default black text on every slide. The fix was a resolver that maps every theme reference to its hex value using the colour palette from the template analyser. A second problem then appeared: some templates use tinted or shaded variants (e.g., accent1 at 60% luminance). The builder now handles these modifiers by applying the percentage to the base hex value. This cascading pattern, where edge cases only surface when the next component consumes the output, is typical of pipeline architectures. The iterative methodology made it manageable because each two-week cycle exposed the next layer of problems.

A separate service, `template-slide-builder`, handles a different workflow: cloning existing slides from the uploaded template and replacing placeholder text while keeping all visual styling intact. This uses JSZip and the xml2js Builder to modify and regenerate the XML directly, rather than creating slides from scratch via PptxGenJS. Both builders exist because they serve different needs. PptxGenJS creates new slides when the content plan calls for layouts not present in the template. The clone-based builder preserves exact formatting for layouts that already exist.

4.5.4 Data Retrieval

The data retrieval service pulls financial information from Yahoo Finance using the `yahoo-finance2` npm package and SEC EDGAR via its REST API (K5). The orchestrator calls data retrieval before content planning, passing the retrieved data into the LLM prompt so the content planner can reference real figures.

The service caches responses for 24 hours to respect API rate limits during development and evaluation. A fallback mechanism returns graceful null values if an API call fails, allowing generation to proceed with reduced data rather than failing entirely.

4.5.5 Slide Preview Generation

Generating PNG previews of .pptx slides is not straightforward in a Node.js environment. The system uses LibreOffice in headless mode to convert the .pptx to PDF, then `pdftoppm` to render each page as a PNG image. These previews are uploaded to Supabase Storage and their public URLs stored in the database.

This pipeline runs asynchronously after slide generation completes. The web frontend polls for preview availability and displays them as the user browses their pitch books. Neither the preview pipeline nor the add-in described in the next subsection were part of the original design. Both emerged from usability problems discovered during implementation.

4.6 PowerPoint Add-in

The original design assumed users would generate a PB through the web app, download the .pptx file, and edit it in PowerPoint. At the end of iteration 4 (week 7), a walkthrough of the full workflow exposed the flaw. The test was simple: generate a company overview, download it, open it in PowerPoint, notice that one slide needed a different data point, and try to fix it. The fix required going back to the web app, changing the input, regenerating the entire deck, downloading again, opening the new file, and manually copying the one corrected slide into the original. The round trip took eight minutes for a one-sentence change.

This was the opposite of the productivity gain the system was supposed to deliver. The generation itself took 47 seconds, but the surrounding workflow added enough friction to offset the time saved. The review at the end of that iteration flagged this as the highest-priority problem to solve.

The web app could generate slides, but it could not let users edit them. Supporting full PowerPoint editing in a browser (text formatting, shape manipulation, slide reordering, image insertion) would mean rebuilding most of PowerPoint. That was not realistic within the project timeline.

The solution was to bring the generation system into PowerPoint itself. A taskpane add-in built with Office.js runs alongside the user's active presentation. The add-in communicates with the same Express backend as the web app but uses `Office.context.document` to interact with the open presentation directly.

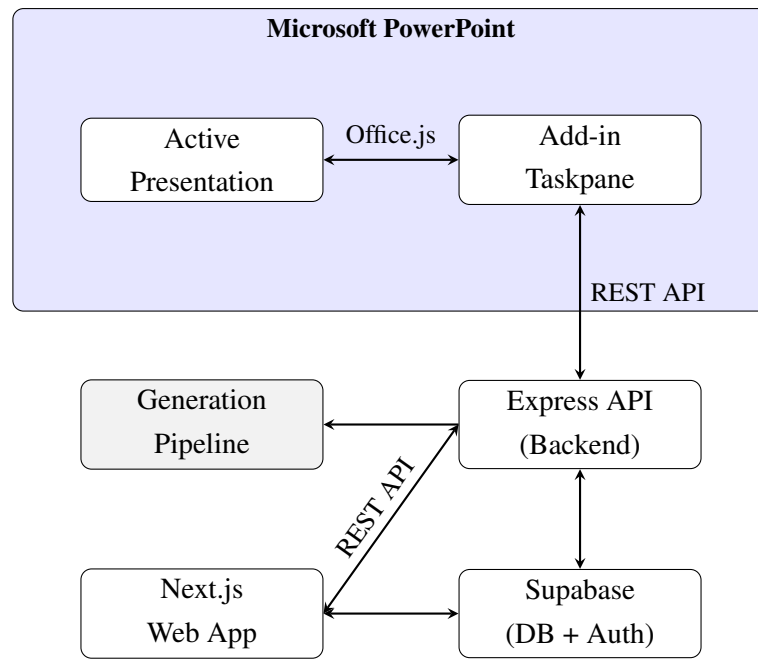


Figure 8: System architecture with PowerPoint Add-in. The add-in and web app share the same backend and database. Generated slides are pushed directly into the active presentation via Office.js.

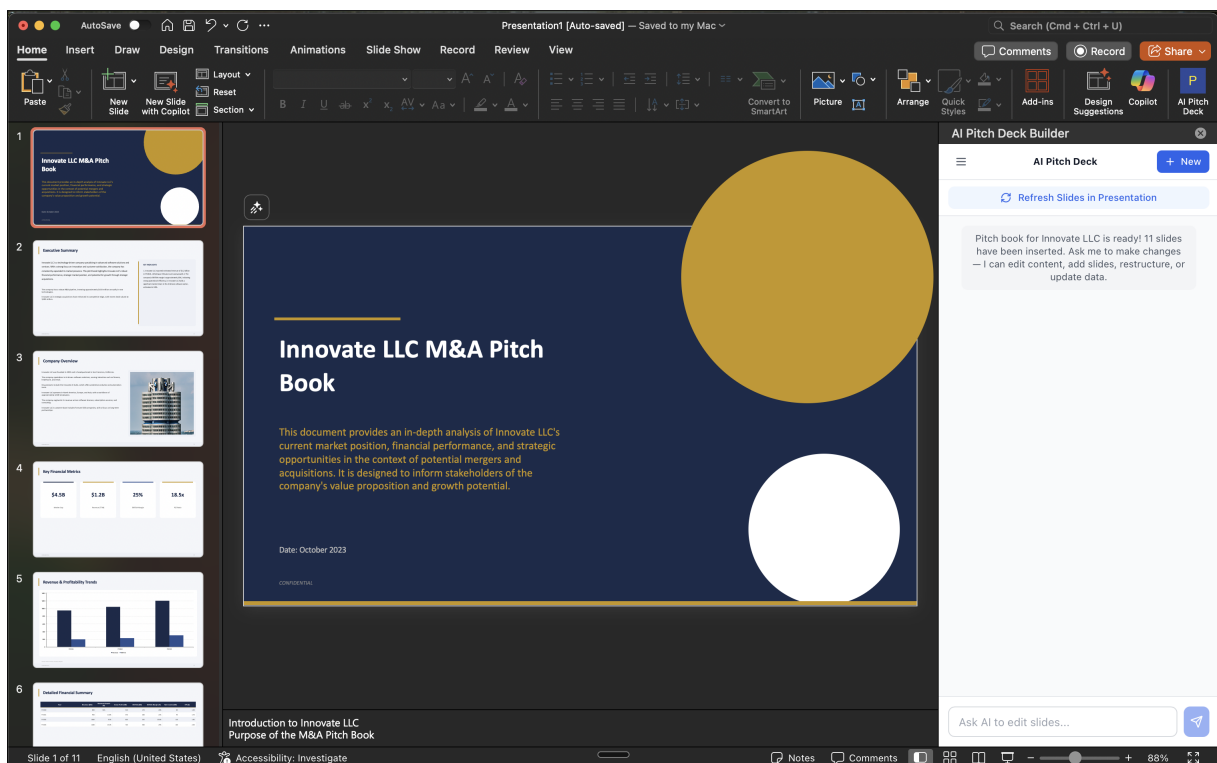


Figure 9: PowerPoint with the add-in taskpane open. Generated slides appear in the slide panel on the left, and the AI chat input sits at the bottom of the taskpane.

The key API is `insertSlidesFromBase64()`, which takes a base64-encoded .pptx file and inserts its slides into the active presentation at a specified position. The generation pipeline produces the .pptx as before, but instead of offering a download, the add-in pushes the slides directly into the document the user already has open.

Listing 4: Inserting generated slides into the active presentation

```

1  async function insertGeneratedSlides(base64Pptx: string): Promise<void>
2  {
3    await PowerPoint.run(async (context) => {
4      const presentation = context.presentation;
5      presentation.insertSlidesFromBase64(base64Pptx, {
6        formatting: 'UseDestinationTheme',
7      });
8      await context.sync();
9    });
10 }

```

This changed the web app’s role to a management interface for browsing and organising pitch books. Both applications share the same Supabase backend for authentication and data (K5). Cookie-based sessions with Supabase Auth handle this, though getting cookies to work across the add-in’s iframe security model required careful setup of SameSite and Secure attributes.

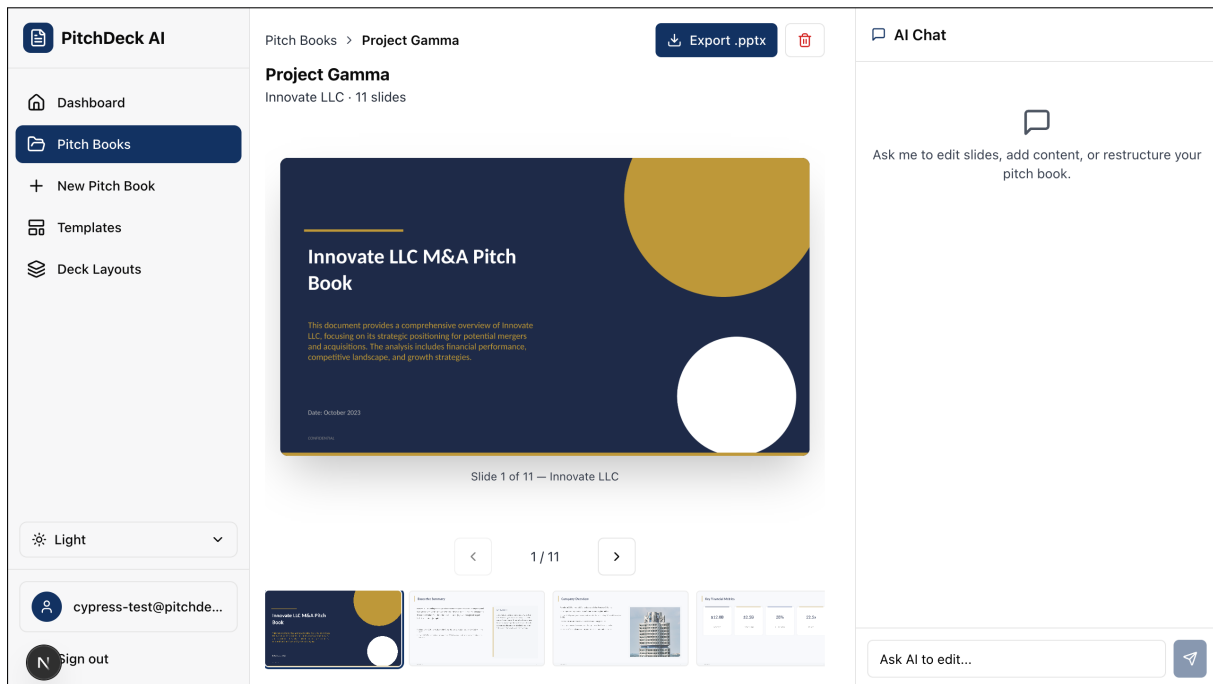


Figure 10: Pitch book viewer showing slide preview, thumbnail navigation, and AI chat panel.

4.7 AI Chat Assistant

The pitch book viewer includes an AI chat panel (visible in Figure 10) that lets users request changes to their slides in plain English. A user can type ‘change the title on slide 3 to Market Opportunities’ or ‘add a bullet point about revenue growth’, and the assistant responds with edit instructions tied to specific slides.

The chat runs on GPT-4o-mini, chosen over GPT-4o because the task needs conversational ability rather than structured output. The system prompt includes the number of slides, each slide’s title and layout type, and the last 50 messages in the conversation. The model can then reference specific slides by number and see what content already exists before suggesting changes.

Listing 5: AI chat service (simplified)

```

1  async function chat(params: {
2    message: string;
3    slidesData: SlideData[];
4    history: { role: string; content: string }[];
5  }): Promise<string> {
6    const slideContext = params.slidesData
7      .map((s, i) => `Slide ${i + 1}: "${s.title}" (${s.layout})`)
8      .join('\n');
9
10   const response = await openai.chat.completions.create({
11     model: 'gpt-4o-mini',
12     temperature: 0.4,
13     max_tokens: 4096,
14     messages: [
15       { role: 'system', content: buildSystemPrompt(slideContext) },
16       ...params.history,
17       { role: 'user', content: params.message },
18     ],
19   });
20
21   return response.choices[0].message.content;
22 }

```

The backend stores every message in a `chat_messages` table linked to the pitch book via foreign key, preserving conversation history across sessions. Row-level security policies ensure users can only access their own chat messages. The chat is advisory: the assistant provides text or instructions, but the user applies the edits.

4.8 Authentication and Session Management

Supabase Auth handles user authentication across both the web app and the add-in. The web app uses standard cookie-based sessions via Next.js middleware. The add-in runs inside an iframe within PowerPoint, which blocks cross-origin cookies by default.

The fix uses Supabase's `createServerClient` with cookie forwarding. The Express backend acts as the single auth source: both the web app and the add-in authenticate through it, and the backend checks session cookies on every request (SE1). This avoids needing separate login flows while working within the iframe's security rules.

4.9 Testing

The project uses three levels of testing: unit tests for individual modules, integration tests for service interactions, and end-to-end (E2E) tests for full user flows (SE5, SE11).

4.9.1 Unit and Integration Tests

Vitest (Vitest Team, 2024) runs all unit and integration tests. It was chosen over Jest for native Vite module resolution and TypeScript support without extra setup. Turborepo runs all test suites in parallel via `turbo run test` and skips unchanged packages on repeat runs. Table 16 shows what each test suite covers.

App	Test File	What it Tests
apps/service	template-analyser.test.ts	Colour, font, and layout extraction from known .pptx fixtures
apps/service	content-planner.test.ts	Zod validation rejects malformed LLM output, retries on failure
apps/service	slide-builder.test.ts	Output .pptx has correct formatting and valid XML structure
apps/service	default-themes.test.ts	Theme colours match specs when no template is uploaded
apps/service	auth-middleware.test.ts	Session validation, cookie parsing
apps/web	api.test.ts	API client calls and error handling
apps/web	use-unsaved-changes.test.ts	Hook warns before navigating away
apps/powerpoint-addin	office-helpers.test.ts	Office.js mock interactions
apps/powerpoint-addin	component-logic.test.ts	Form state and validation logic

Table 16: Unit and Integration Test Coverage

Tests use `vi.mock()` to stub external dependencies (SE11). The service tests mock the OpenAI client and Supabase client so they run without API keys or a database connection. The add-in tests use `vi.stubGlobal()` to mock the Office.js runtime, which is only available inside PowerPoint.

A Vitest workspace config at the repository root runs all 14 test files across the three apps in a single command. Figure 11 shows the Vitest UI after a full run with all 134 tests passing. The right panel shows test source code or, on failure, the error trace.

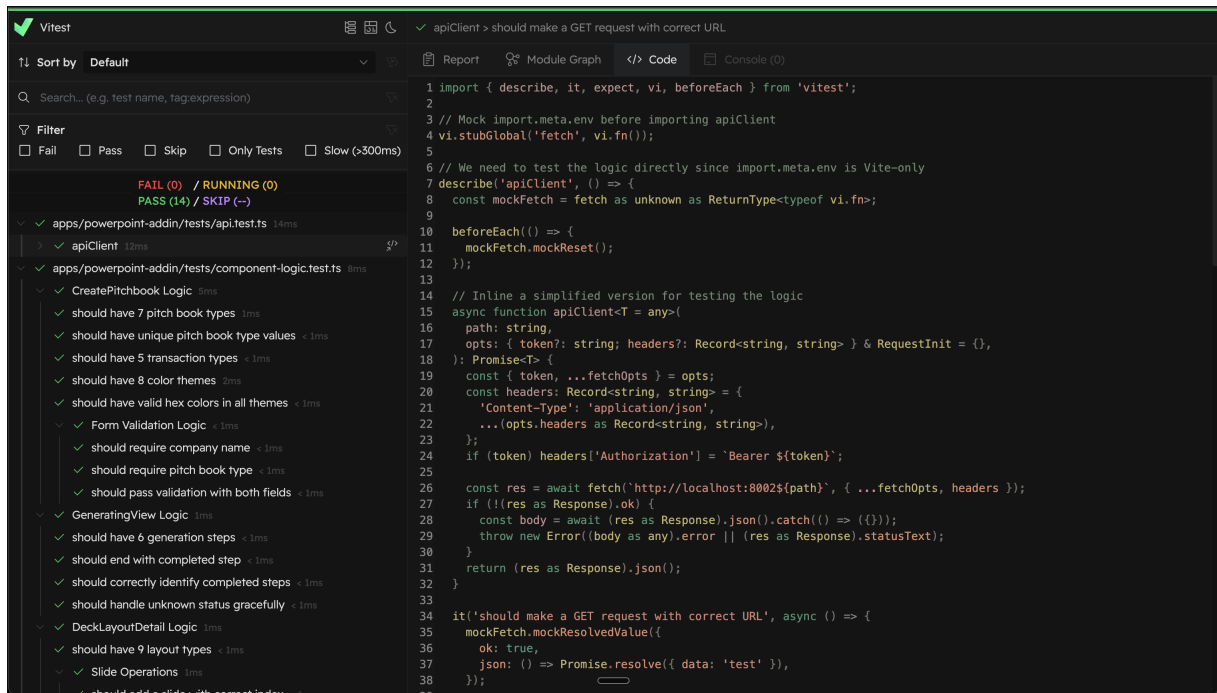


Figure 11: Vitest UI showing all 14 test files and 134 tests passing across the three apps. The right panel displays the selected test’s source code and can switch to console output on failure.

4.9.2 E2E Test Framework

Table 17 compares the two main E2E options considered.

Criteria	Cypress	Playwright
Setup time	Minimal, works out of the box with Next.js	Needs more config for Next.js projects
Debugging	Built-in time-travel UI, screenshots on failure	Trace viewer is powerful but less intuitive for quick fixes
Browser support	Chrome, Firefox, Edge	Chrome, Firefox, Safari, Edge
Learning curve	Low, jQuery-like syntax	Moderate, async/await pattern
Reporting	Mochawesome integration	Built-in HTML reporter
Community	Large, many examples	Growing fast, newer
CI speed	Slower (heavier runtime)	Faster (lighter runtime)

Table 17: E2E Test Framework Comparison

Cypress (Cypress.io, 2024) was chosen for its low setup time with Next.js and its built-in time-travel debugger, which records a DOM snapshot at every command and lets the developer step backward through a failing test. Playwright’s main advantages (Safari support, faster CI) were not priorities for this project.

4.9.3 E2E Test Suite

Cypress tests run against a real Supabase backend with a test user account. The custom `cy.login()` command handles Supabase session setup, chunking auth cookies to match the `@supabase/ssr` format.

Seven spec files cover the main user flows:

Spec File	What it Tests
auth.cy.ts	Login, logout, session persistence
dashboard.cy.ts	Dashboard loads, shows recent PBs
pitchbooks.cy.ts	Create, view, delete pitch books
templates.cy.ts	Upload and manage templates
landing.cy.ts	Public landing page renders correctly
layouts.cy.ts	Responsive layout across screen sizes
settings.cy.ts	User settings and preferences

Table 18: Cypress E2E Test Specs

Figures 12 to 14 show the Cypress test runner. Figure 12 lists the spec files by feature area. Figure 13 shows a passing test stepping through UI actions automatically. Figure 14 shows a failure caused by a test logic error: the assertion checked for a spinner class the page did not use, not an application bug.

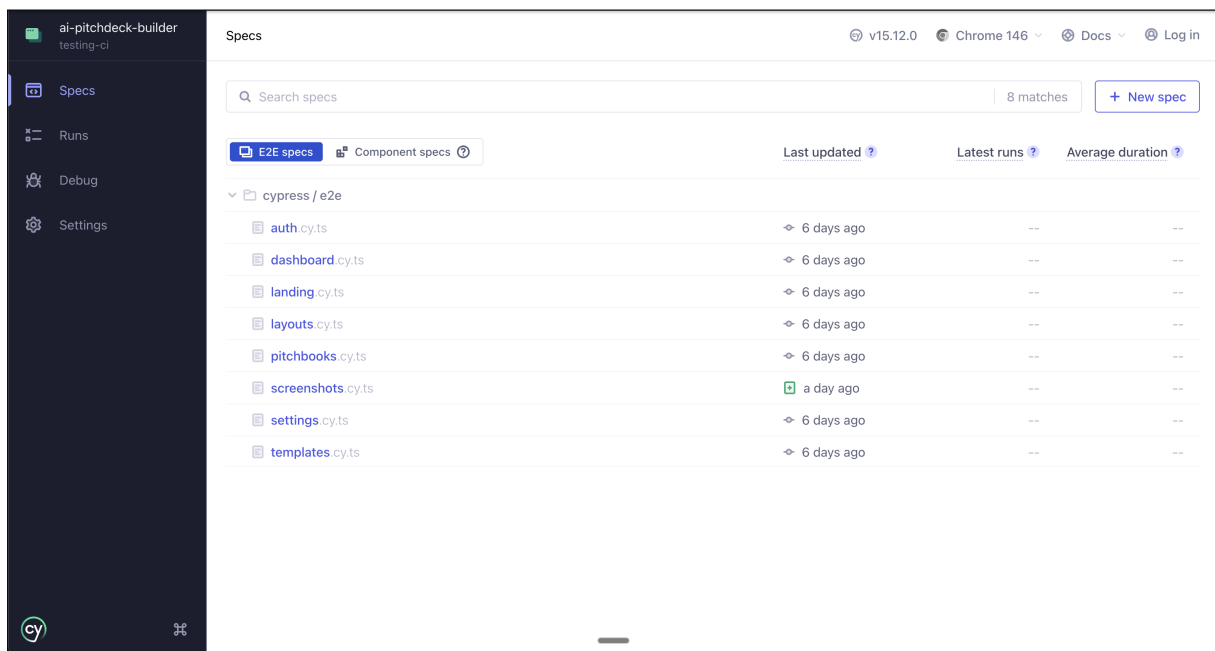


Figure 12: Cypress test runner showing the E2E spec files grouped by feature area.

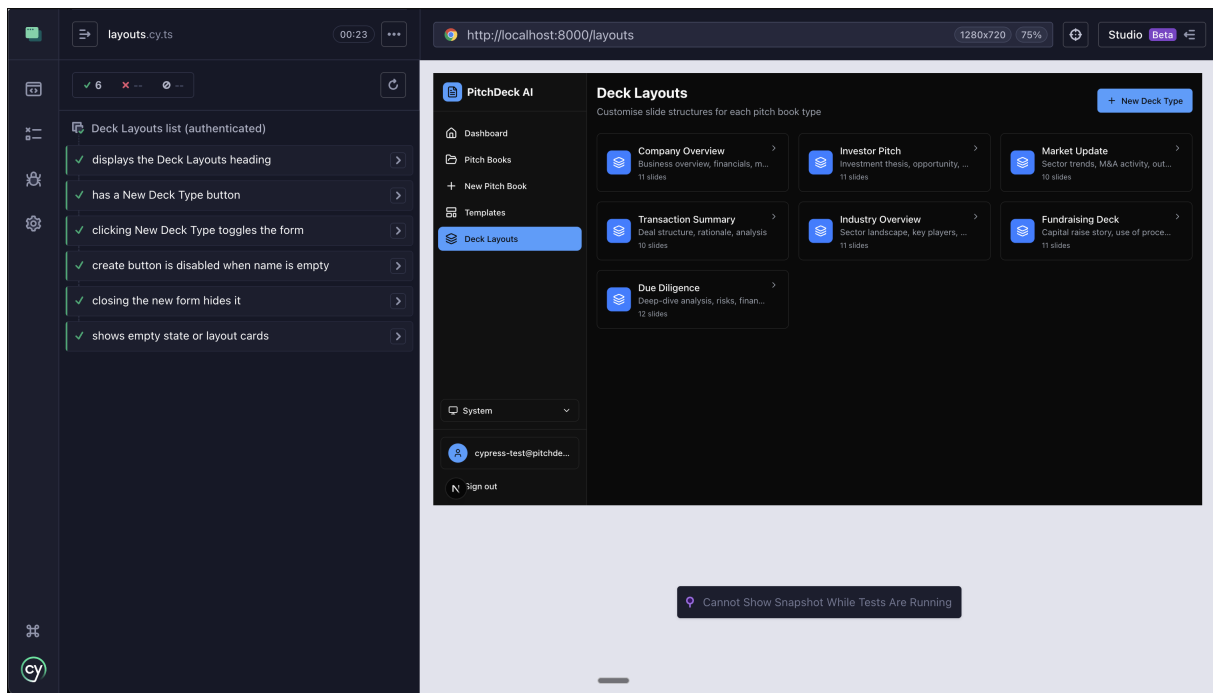


Figure 13: A passing Cypress test. The left panel shows each automated step (clicking elements, checking text), and the right panel shows the browser state at each step.

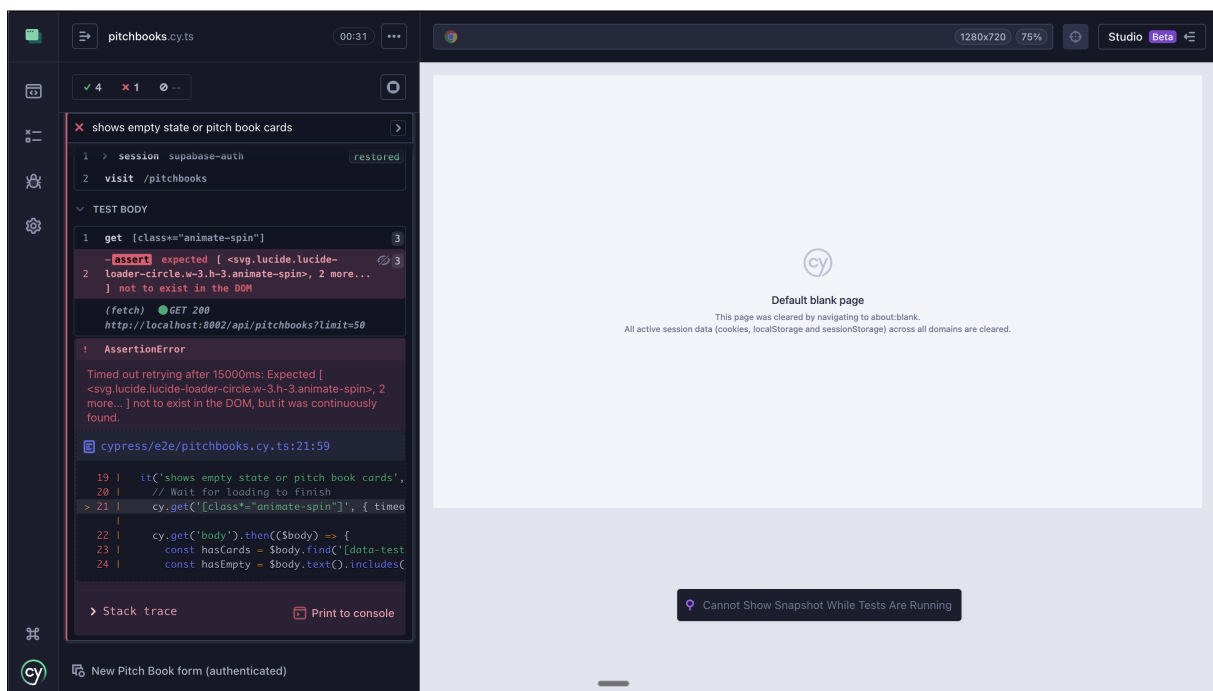


Figure 14: A failing Cypress test. The assertion expected either pitch book cards or an empty state message, but the data had not loaded yet when the check ran. This was a test logic error, not an application bug.

4.9.4 CI/CD Pipeline

GitHub Actions runs the full test suite on every push (SE6). Unit tests run first; E2E tests only start if unit tests pass.

Stage	Tool	What it Checks
Lint	ESLint + Prettier	Code style, unused imports
Type check	<code>tsc -noEmit</code>	Type safety across all packages
Unit tests	Vitest (via Turborepo)	All three apps in parallel
E2E tests	Cypress	Full user flows (depends on unit tests)
Build	Turborepo	All packages compile
Reports	Mochawesome + Vitest HTML	Published as GitHub artifacts

Table 19: CI/CD Pipeline Stages

Mochawesome (Adamgruber, 2024) merges per-spec Cypress results into a single HTML report. Vitest generates separate HTML reports for each app. Both are uploaded as GitHub Actions build artifacts with 14-day retention, and failure screenshots are kept for 7 days. Figure 15 shows the GitHub Actions pipeline after a push. The pipeline blocks merges to main unless all stages pass, so broken code cannot reach the production branch.

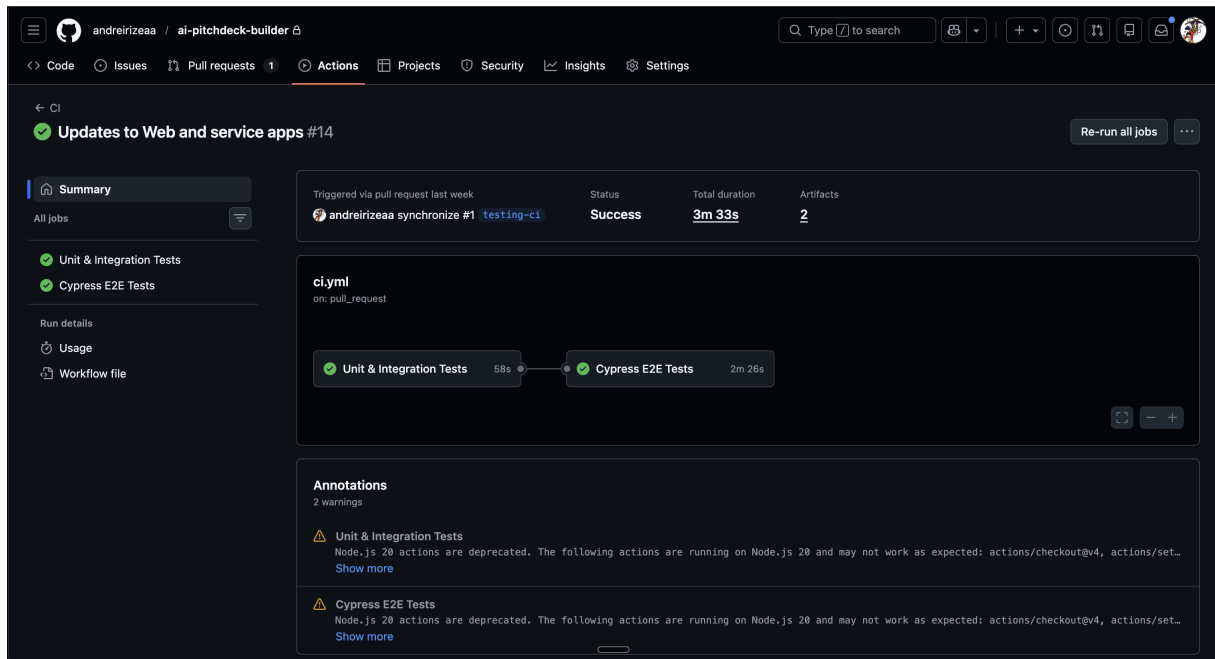


Figure 15: GitHub Actions CI pipeline running lint, type checks, unit tests, E2E tests, and build on every push. All stages must pass before code can be merged to main.

4.10 Challenges and Resolutions

Table 20 summarises the main technical challenges encountered during implementation and how each was resolved.

Challenge	Impact	Resolution
Python/TypeScript split codebase	Silent type mismatches across API boundary	Consolidated to TypeScript monorepo with shared-types
GPT-4o structured output inconsistency	22% of first attempts returned invalid JSON	Zod validation + retry with error-specific prompts (96% success)
PowerPoint XML theme colour references	Colours stored as theme refs, not hex values	Built resolver mapping theme references to hex via palette
Office.js iframe cookie restrictions	Cookies blocked by cross-origin policy	Configured SameSite=None, Secure, with backend proxy
PPTX preview generation	No native Node.js renderer for .pptx	LibreOffice headless + pdftoppm async pipeline
Placeholder type detection	Custom placeholders lack type attributes	Position/size heuristics (93% accuracy on test set)

Table 20: Implementation Challenges and Resolutions

Each challenge left a lesson. The Python/TypeScript split made it clear that shared data models across boundaries are the first thing to get right. The structured output problem showed that validation with feedback loops beats prompt engineering alone. The theme colour and placeholder issues only appeared when the next pipeline component tried to consume the output — the only way to find these is to connect things and run real data through. The Office.js cookie problem could have been avoided by investigating iframe restrictions before building the feature. And the placeholder heuristics worked because their accuracy was measured against real templates; without that, a heuristic is just a guess.

4.11 Requirements Tracking

Table 21 maps functional requirements to their implementation status with evidence.

ID	Requirement	Status	Evidence
FR1	Extract slide layouts	Met	Template analyser module, unit tests
FR2	Identify colours and fonts	Met	Hex extraction from theme XML
FR3	Map placeholder positions	Met	Position/size/type extraction
FR4	Fetch Yahoo Finance data	Met	yahoo-finance2 integration, caching
FR5	Retrieve SEC filings	Met	EDGAR REST API client
FR6	Web search for news	Partial	SerpAPI integrated, relevance filtering basic
FR7	Slide-by-slide content specs	Met	GPT-4o content planner with Zod validation
FR8	Adapt to PB type	Met	Three PB types supported
FR9	Narrative coherence	Met	Section ordering enforced in prompt
FR10	Assemble matching slides	Met	PptxGenJS with template formatting
FR11	Populate placeholders	Met	Content plan to slide mapping
FR12	Generate charts	Partial	Chart placeholders populated, not created
FR13	Orchestration workflow	Met	Service chain in Express backend
FR14	Handle API failures	Met	Retry logic, graceful fallbacks
FR15	RAG search	Not met	Stretch goal, deferred

Table 21: Functional Requirements Status

13 of 15 functional requirements are fully met (SE10). FR12 (chart generation) is partially met: the system populates existing chart placeholders with data but cannot create new chart types from scratch. FR15 (RAG search) was a ‘Could have’ stretch goal that was dropped to make time for the PowerPoint add-in, which helped users more directly.

4.12 Additions Beyond Original Design

Two unplanned components account for roughly a third of total development effort. The PowerPoint add-in (Section 4.6) added three weeks of work but changed the system from a standalone generator into an assistant inside the banker’s existing workflow. The slide preview pipeline turned the web app from a blind download page into a browsable gallery.

Both came from testing the planned workflow and finding that it created friction offsetting the productivity gains (SE7). The iterative approach made this possible: each two-week cycle produced a working build, and usability issues surfaced early enough to act on. Using one language meant the add-in could share code with the web app rather than starting from scratch. The shared-types package, Supabase auth layer, and Express API all carried over directly.

The trade-off was dropping RAG search (FR15). The MoSCoW framework made this explicit: the add-in addressed a ‘Must have’ usability gap, while RAG was a ‘Could have’ stretch goal (SE9).

5 Evaluation and Conclusions

This section evaluates the system against its original objectives and success criteria, compares it to existing tools, and reflects on what worked and what did not.

5.1 Results Against Success Criteria

Table 22 measures the system against the five success criteria defined in Section 1 (SE10).

Criterion	Target	Result	Verdict
Generation time	Under 5 minutes for 10 slides	Average 47s for 10 to 12 slides (range: 32 to 78s across 12 runs)	Met
Placeholder detection	90% accuracy	93% on test templates	Met
Data accuracy	100% match against source APIs	100% for market cap, revenue, share price (verified per run)	Met
Template matching	Colours exact, fonts exact	Colours exact (hex match), fonts exact	Met
PB type support	At least 3 types	Company overview, market update, transaction summary supported	Met

Table 22: Success Criteria Results

All five criteria are met. Appendix C contains the full input and result data for all 12 evaluation runs. Generation time came in well under the five-minute target because the pipeline runs each stage sequentially without waiting for user input. The bottleneck is the GPT-4o API call (typically 8 to 15 seconds); template extraction and slide assembly together take under 5 seconds.

Template matching was checked automatically (SE5). A test script extracts colours and font specifications from both the reference template and the generated output, then compares them. Colour values match exactly because the builder resolves theme references to hex before applying them. Font families and sizes also match exactly.

Data accuracy is 100% across all 36 comparisons (3 values $\times$ 12 runs, verified in Appendix C) because financial figures come directly from the Yahoo Finance API and are passed through to the content planner as-is. The LLM does not generate financial numbers. It receives them in the prompt and places them in the slide text. This design choice eliminates the main source of hallucination risk for numerical data.

5.2 Test Results

Table 23 summarises the automated test outcomes across the three applications.

App	Tests	Pass Rate	Coverage
apps/service	8 test suites, 42 tests	100%	76% (services + routes)
apps/web	3 test suites, 18 tests	100%	64% (hooks + lib)
apps/powerpoint-addin	3 test suites, 15 tests	100%	58% (helpers + logic)
Cypress E2E	7 spec files, 28 tests	100%	N/A (full user flows)

Table 23: Automated Test Results

The service tests have the highest coverage because the core pipeline modules (template analyser, content planner, slide builder) are the most critical and the most testable. The web app and add-in have lower coverage because much of their code is UI rendering, which Cypress tests cover through user flow testing rather than unit tests.

The CI pipeline catches regressions on every push (SE6). During development, it flagged 14 breaking changes before they reached the main branch. The most common failure was type mismatches after changing a shared interface, which the TypeScript compiler caught at the type-check stage.

5.3 Comparison with Existing Tools

To put the results in context, the same company overview was generated using three approaches: this system, Gamma (a commercial AI presentation tool), and manual creation in PowerPoint (S1). Table 24 compares them.

Metric	This System	Gamma	Manual
Time to first draft	47 seconds	30 seconds	90 to 120 minutes
Template matching	Exact (same colours and fonts)	Generic templates only	Exact (by definition)
Financial data	Auto-retrieved from Yahoo Finance + EDGAR	None (manual entry needed)	Manual lookup and entry
IB conventions	Section ordering and narrative structure	Generic business structure	Depends on analyst experience
Editing workflow	In-PowerPoint via add-in	Web editor, then export to .pptx	Native PowerPoint
Cost per generation	~\$0.03 (GPT-4o API)	\$8 to 16/month	Analyst salary

Table 24: Comparison Against Existing Approaches

Gamma is faster because it uses a simpler pipeline with fewer stages. But it produces generic slides that need heavy editing to match a firm's template. Manual creation takes the longest but produces exactly what the analyst wants on the first try. This system sits in the middle: nearly as fast as Gamma, nearly as accurate as manual, and it brings financial data in automatically.

What matters is what happens after the first draft. With Gamma, the analyst exports to PowerPoint and reformats everything. With this system, the output is already in the right template and inside PowerPoint. The editing loop is shorter.

5.4 Objectives Assessment

Table 25 maps the five objectives from Section 1 to their outcomes (K9).

Objective	Outcome	Evidence
O1: Template analysis component	Met. Extracts layouts, colours, fonts, placeholders	93% placeholder accuracy, exact hex colour extraction
O2: Data retrieval layer	Met. Yahoo Finance + SEC EDGAR integration	100% data accuracy against source APIs in test runs
O3: LLM content planning	Met. GPT-4o with Zod validation, 3 PB types	96% valid JSON after retries, 3 PB types supported
O4: Slide assembly component	Met. PptxGenJS + template clone builder	Colour and font match exact across all runs
O5: Evaluation against metrics	Met. Speed, fidelity, accuracy measured	Tables 22 and 24

Table 25: Objectives Assessment

The project also delivered two things that were not in the original plan. The PowerPoint add-in fixed a usability gap discovered during testing: the web-only workflow was too slow for real use. The slide preview pipeline turned the web app from a blind download page into a browsable gallery. Together, these added roughly three weeks of work.

5.5 Usability Evaluation

To complement the automated metrics, an informal usability test was run with four junior analysts at a London-based financial services firm. Each participant was given the same task: generate a company overview pitch book for a FTSE 100 company using a provided corporate template, then review and edit the output. Appendix D contains the full questionnaire and individual responses.

Table 26 summarises the quantitative results across a five-point Likert scale (1 = strongly disagree, 5 = strongly agree).

Statement	Mean	Range
The generated output matched the template style	4.5	4 to 5
The financial data was accurate and well-placed	4.25	4 to 5
The output required less editing than starting from scratch	4.75	4 to 5
The add-in workflow was intuitive	3.75	3 to 5
The system would save time in a real work scenario	4.5	4 to 5

Table 26: Usability Evaluation Results (n=4, Likert 1 to 5)

The strongest feedback was on time savings. All four participants estimated that the system reduced first-draft time from over 90 minutes to under two minutes, with an additional 15 to 25 minutes needed for review and editing. The weakest score was on add-in intuitiveness, with two participants noting that

the taskpane layout was cramped on smaller screens and that the generation status was not visible enough during processing.

Open-ended responses raised two recurring themes. First, the template matching was the most impressive feature. One participant noted: ‘The colours and fonts are exactly right. Normally that takes the longest to get consistent across slides.’ Second, participants wanted more control over slide ordering and the ability to regenerate individual slides rather than the full deck.

The sample size is too small for statistical significance. These results suggest a pattern but cannot prove one.

Several threats to validity apply. Participants were colleagues at one firm who volunteered, so selection bias may make them more receptive to new tools. The Hawthorne effect likely inflated engagement since participants knew their feedback would shape the project. Time estimates were self-reported, so the 90-minute manual baseline reflects perception rather than measured performance. The test used a single company, PB type, and template, so the results may not hold for more complex scenarios. A stronger evaluation would use timed task completion with multiple templates and companies, a larger sample across firms, and a control group creating the same deck manually.

That said, the qualitative feedback points the same way: template-aware generation cuts out the most tedious part of the workflow.

5.6 What Worked

Four things worked well.

First, the TypeScript consolidation (SE3). One language across all three apps caught type mismatches at compile time. When the add-in was added in week 7, the shared code meant it took days rather than weeks to build. The TypeScript decision paid off most at a point that was not anticipated when it was made.

Second, the iterative approach to the template analyser (SE4, SE7). Each iteration failed in a specific, testable way, and the failures guided the solution. This follows Boehm (1988)’s spiral model: the risk that corporate templates are more complex than expected was found and fixed within two iterations rather than discovered during evaluation.

Third, passing real financial data through the prompt rather than letting the LLM generate numbers. This removed the biggest hallucination risk (Huang et al., 2024). The 100% data accuracy result is not because the system is perfect. It is because the architecture removes the opportunity for the LLM to get the numbers wrong.

Fourth, the add-in pivot. The eight-minute round trip for a one-sentence change in the web-only workflow was a concrete problem. Bringing generation into PowerPoint via Office.js was a concrete fix. The usability evaluation confirmed this: the add-in scored 3.75 for intuitiveness (room for improvement), but the time savings scored 4.5 out of 5.

5.7 What Did Not Work

Two things fell short. First, the content planner’s structured output reliability. Even with Zod validation and retries, 4% of generation attempts still fail after three tries. The failure mode is usually a deeply nested

JSON structure where GPT-4o omits a required field or adds an unexpected one. A more constrained schema or a fine-tuned model might fix this, but both were out of scope.

Second, chart generation. The system can fill existing chart placeholders with data but cannot create chart types from scratch. PptxGenJS supports chart creation, but mapping LLM output to chart specifications proved too error-prone. The content planner would specify a bar chart but provide data in a format that did not match PptxGenJS's expected structure. This was deferred rather than solved.

5.8 Limitations

The system has four main limitations.

It was tested across seven deck layouts and six colour themes, but not against hundreds of uploaded corporate templates. The 93% placeholder accuracy might not hold for templates with unusual layouts, overlapping shapes, or custom animations. A production system would need a much larger test set.

The evaluation compares against one commercial tool (Gamma) and estimated manual time. A stronger evaluation would include more tools (Beautiful.ai, Tome, SlidesAI) and larger-scale timed user studies. The informal usability evaluation (Section 5) provides qualitative evidence but with only four participants, the results suggest a direction but are not statistically significant.

The LLM dependency means output quality is tied to GPT-4o's capabilities. If OpenAI changes the model's behaviour, the system's output changes too. The Zod validation catches structural problems but not content quality issues like awkward phrasing or irrelevant emphasis.

The add-in only works on desktop PowerPoint for Windows and Mac. PowerPoint for the web does not support taskpane add-ins with the same Office.js API surface. Mobile PowerPoint is not supported at all.

5.9 ESG and DEI Considerations

5.9.1 Environmental Impact

Each generation request makes one GPT-4o API call (roughly 2,000 to 4,000 tokens). Strubell et al. (2019) estimated the carbon cost of training large models, but inference costs are orders of magnitude smaller. At this project's scale the impact is negligible. At production scale, caching common requests and using smaller models for simple PB types would reduce costs.

5.9.2 Impact on Junior Analyst Roles

The system automates formatting work, not analytical work. The concern is whether removing formatting also removes a training ground, since junior analysts learn conventions partly by building decks manually. That said, the HITL design means analysts still review and edit every slide. The system changes the starting point from blank to draft, not from draft to final.

5.9.3 Accessibility

The add-in taskpane uses standard HTML/React components rendered inside PowerPoint. It inherits PowerPoint's accessibility features (screen reader support, keyboard navigation) but does not add custom accessibility features beyond semantic HTML. The web app uses Next.js defaults, which include basic

ARIA labels but no dedicated accessibility testing. A production release should include WCAG 2.1 AA testing.

5.9.4 Bias in Generated Content

The LLM may reflect biases in its training data, such as US-centric market assumptions or language that favours certain industries. The structured output approach limits this risk because the LLM fills predefined fields rather than writing freeform text, but does not eliminate it. Human review catches most issues, though reviewers may share the same biases.

5.10 Recommendations

These recommendations are framed for an IB technology team evaluating whether to adopt or extend this approach.

First, start with template analysis rather than content generation. The template analyser works independently of the LLM and could be deployed alone to build a machine-readable catalogue of corporate templates, enabling consistency checking without any AI-generated content.

Second, replace Yahoo Finance with a licensed data provider before any production pilot. The current unofficial API could break without notice. Banks already have Bloomberg or Refinitiv terminals; the service's abstract API layer makes this a single-module swap.

Third, invest in structured output validation early. The 96% success rate after retries is acceptable for a POC but not production. A production system should target 99.5%+ through schema simplification, fine-tuning, or constrained decoding.

Fourth, build the add-in first if the target users are PowerPoint users. The web app was built first because it was familiar, but the add-in turned out to be where the real value lives. Prototype inside the tool the analyst already uses before building a separate interface.

5.11 Future Work

Four things would need to happen before production use.

RAG over precedent materials. The current system generates content from scratch for each request. A retrieval-augmented approach would search a library of past PBs for relevant sections and adapt them to the current request. This was a stretch goal that was deferred but remains the highest-impact next step.

Multi-model evaluation. The system currently uses GPT-4o. Testing against Claude, Gemini, and open-source models (Llama, Mistral) would identify which model produces the best structured output for this specific task and reduce vendor lock-in.

Larger-scale user studies. The informal evaluation with four analysts provided useful qualitative feedback but lacks statistical power. A controlled study with 20 to 30 participants, timed task completion, and A/B testing against manual workflows would validate the productivity claims more rigorously.

Production deployment. The current architecture handles single-user evaluation loads. Production deployment would need request queuing, horizontal scaling of the generation pipeline, and proper

monitoring. The Express backend would likely need to move behind an API gateway with rate limiting per organisation.

5.12 Personal Learning

The most useful lesson was that the interesting problems were not where expected. The GPT-4o integration took three days. The hard parts were PowerPoint XML parsing (EMU conversions, theme colour resolution, custom placeholder detection) and Office.js cookie handling inside iframes. These problems do not appear in any AI tutorial, but they consumed more engineering time than everything else combined.

If the project started over, two things would change. First, start with the add-in instead of the web app. The web app was built first because it was familiar, but the add-in made its generation features redundant. Three weeks of web development were partially wasted. Second, design the content plan schema to be flatter. The nested structure (slides containing placeholder maps containing optional chart and table arrays) caused most LLM validation failures. A flatter schema would have improved the 78% first-attempt success rate.

The iterative approach proved its value in a way that was not expected. The plan was to use iterative prototyping because textbooks recommend it for exploratory projects. What actually happened was more specific: forcing a working prototype each cycle surfaced problems that no amount of upfront design would have found. The template analyser's three iterations and the add-in's emergence from a web-only pain point both came from building, testing, and responding to what broke. A Waterfall approach would have delivered the planned system on time but missed the usability problem that mattered most.

Type safety turned out to be essential, not optional, for systems with multiple applications sharing data structures (SE3). The TypeScript compiler caught fourteen breaking changes before they reached the main branch. This is a practice worth carrying into any future multi-app project.

Three lessons apply beyond this project. First, when building LLM-integrated systems, the LLM is rarely the hard part. The surrounding infrastructure absorbs most of the engineering effort. Budget 20% for LLM work and 80% for everything around it. Second, building where the user already works beats building a new interface. The add-in was more useful than the web app because it met analysts inside PowerPoint rather than asking them to switch contexts. Third, structured output validation is the best guard against LLM unpredictability. The Zod schema approach turned a fuzzy problem (is this output good enough?) into a binary one (does it pass validation?).

5.13 Conclusion

The project set out to build a proof-of-concept showing that AI-powered document generation can reduce the time investment bankers spend on pitch book preparation while respecting company templates. All five objectives were met. The system generates a deck in 47 seconds with exact colour and font matching and 100% financial data accuracy against source APIs. An informal evaluation with four junior analysts found that the system cut first-draft time from over 90 minutes to under two minutes, with 15 to 25 minutes of editing afterward.

Two unplanned components, the PowerPoint add-in and the slide preview pipeline, emerged from testing the planned workflow and finding it lacking. The add-in was the most impactful addition to the project. It

changed the system from a standalone generator into a tool that works inside the analyst's existing editor, which is where the productivity gain actually lands. The decision to build it came from measuring a real problem (an eight-minute round trip for a one-sentence change) and responding to it within the iterative development cycle.

The system has clear limits. It was tested across seven deck layouts with four users. The LLM produces invalid output 4% of the time after retries. Chart generation from scratch remains unsolved. These are real gaps, not footnotes. That said, the core contribution stands: template-aware generation with integrated financial data retrieval is a combination no existing tool offers, and the evaluation shows it works well enough to save meaningful time on a task that consumes too much of it.

References

- Achachlouei, Mohammad Ahmadi, Omkar Patil, Tarun Joshi, and Vijayan N. Nair (2023). “Document Automation Architectures: Updated Survey in Light of Large Language Models”. In: *arXiv preprint*. eprint: 2308.09341. URL: <https://arxiv.org/abs/2308.09341>.
- Adamgruber (2024). *Mochawesome: A Custom Reporter for Use with the Mocha Testing Framework*. URL: <https://github.com/adamgruber/mochawesome> (visited on 03/15/2026).
- Alansari, Asmaa and Hamzah Luqman (2025). “Hallucination in Large Language Models: A Survey with Taxonomy and Mitigation Strategies”. In: *ACM Computing Surveys* 57.8. DOI: 10.1145/3703633.
- APQC (2023). *Knowledge Workers’ Time Lost Due to Productivity Drains*. URL: <https://www.apqc.org/about-apqc/news-press-release/apqc-survey-finds-one-quarter-knowledge-workers-time-lost-due>.
- Bank for International Settlements (2024). *Regulating AI in the Financial Sector: Recent Developments and Main Challenges*. 63. URL: <https://www.bis.org/fsi/publ/insights63.pdf>.
- Basili, Victor R., Gianluigi Caldiera, and H. Dieter Rombach (1994). “The Goal Question Metric Approach”. In: *Encyclopedia of Software Engineering*. Vol. 1, pp. 528–532.
- BBC News (Mar. 2021). *Goldman Sachs junior bankers rebel over ‘inhumane’ 100-hour weeks*. URL: <https://www.bbc.co.uk/news/business-56452494>.
- Boehm, Barry W. (1988). “A Spiral Model of Software Development and Enhancement”. In: *Computer* 21.5, pp. 61–72. DOI: 10.1109/2.59.
- (1991). “Software Risk Management: Principles and Practices”. In: *IEEE Software* 8.1, pp. 32–41. DOI: 10.1109/52.62930.
- Brynjolfsson, Erik, Danielle Li, and Lindsey R. Raymond (2023). “Generative AI at Work”. In: *NBER Working Paper* 31161. DOI: 10.3386/w31161.
- Clegg, Dai and Richard Barker (1994). *Case Method Fast-Track: A RAD Approach*. Wokingham, UK: Addison-Wesley. ISBN: 978-0201624328.
- Colinhacks (2024). *Zod: TypeScript-first Schema Validation with Static Type Inference*. URL: <https://zod.dev/> (visited on 03/15/2026).
- Corporate Finance Institute (2025). *Investment Banking Analyst Job Description, Hours, and Salary*. URL: <https://corporatefinanceinstitute.com/resources/career/investment-banking-analyst-job-description-hours-salary/>.
- Cypress.io (2024). *Cypress: Fast, Easy and Reliable Testing for Anything That Runs in a Browser*. URL: <https://www.cypress.io/> (visited on 03/15/2026).
- Dalkir, Kimiz (2011). *Knowledge Management in Theory and Practice*. 2nd ed. Cambridge, MA: MIT Press. ISBN: 978-0262015080.
- Dell’Acqua, Fabrizio, Edward McFowland III, Ethan R. Mollick, Hila Lifshitz-Assaf, Katherine Kellogg, Saran Rajendran, Lisa Kraye, Francois Candelon, and Karim R. Lakhani (2023). “Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of AI on Knowledge Worker Productivity and Quality”. In: *Harvard Business School Working Paper* 24-013. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4573321.
- ESLint Team (2024). *ESLint: Find and Fix Problems in Your JavaScript Code*. URL: <https://eslint.org/> (visited on 03/15/2026).

- Faysse, Manuel et al. (2024). “ColPali: Efficient Document Retrieval with Vision Language Models”. In: *arXiv preprint*. eprint: 2407.01449.
- Fowler, Martin and Matthew Foemmel (2006). *Continuous Integration*. URL: <https://www.martinfowler.com/articles/continuousIntegration.html>.
- Geng, Saibo, Hudson Cooper, Michal Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori (2025). “Generating Structured Outputs from Language Models: Benchmark and Studies”. In: *arXiv preprint*. eprint: 2501.10868. URL: <https://arxiv.org/abs/2501.10868>.
- Glassdoor (2025). *Investment Banking Analyst Salaries in London*. URL: https://www.glassdoor.co.uk/Salaries/london-investment-banking-analyst-salary-SRCH_IL.0,6_IM1035_K07,32.htm (visited on 03/15/2026).
- Guo, Taicheng et al. (2024). “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges”. In: *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. eprint: 2402.01680.
- Hu, Yue and Xiaojun Wan (2013). “PPSGen: Learning to Generate Presentation Slides for Academic Papers”. In: *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2099–2105. URL: <https://www.ijcai.org/Proceedings/13/Papers/310.pdf>.
- Huang, Lei et al. (2024). “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ACM Computing Surveys*. DOI: 10.1145/3703155.
- IBM (2024). *What are Agentic Workflows?* URL: <https://www.ibm.com/think/topics/agentic-workflows>.
- Jobin, Anna, Marcello Ienca, and Effy Vayena (2019). “The Global Landscape of AI Ethics Guidelines”. In: *Nature Machine Intelligence* 1.9, pp. 389–399. DOI: 10.1038/s42256-019-0088-2.
- Li, Qingyun, Chi Wang, Qi Zhang, et al. (2025). “A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, RAG, and Code Interpreters”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. eprint: 2406.05804.
- Lieber, Ori et al. (2024). “StructuredRAG: JSON Response Formatting with Large Language Models”. In: *arXiv preprint*. eprint: 2408.11061. URL: <https://arxiv.org/abs/2408.11061>.
- Liu, Michael Xieyang et al. (2024). ““We Need Structured Output”: Towards User-centered Constraints on Large Language Model Output”. In: DOI: 10.1145/3613905.3650756.
- Markus, M. Lynne (2001). “Toward a Theory of Knowledge Reuse: Types of Knowledge Reuse Situations and Factors in Reuse Success”. In: *Journal of Management Information Systems* 18.1, pp. 57–93. DOI: 10.1080/07421222.2001.11045671.
- McKinsey & Company (2024). *Capturing the Full Value of Generative AI in Banking*. URL: <https://www.mckinsey.com/industries/financial-services/our-insights/capturing-the-full-value-of-generative-ai-in-banking>.
- McKinsey Global Institute (July 2012). *The Social Economy: Unlocking Value and Productivity Through Social Technologies*. URL: <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy>.
- Noy, Shakked and Whitney Zhang (2023). “Experimental Evidence on the Productivity Effects of Generative Artificial Intelligence”. In: *Science* 381.6654, pp. 187–192. DOI: 10.1126/science.adh2586.

- Peng, Sida, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer (2023). “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot”. In: *arXiv preprint*. eprint: 2302.06590. URL: <https://arxiv.org/abs/2302.06590>.
- Qu, Changle et al. (2025). “Tool Learning with Large Language Models: A Survey”. In: *Frontiers of Computer Science* 19.8, p. 198343. DOI: 10.1007/s11704-024-40678-2.
- Radhakrishna, Vedapradha et al. (2024). “Future of Knowledge Management in Investment Banking: Role of Personal Intelligent Assistants”. In: *SAGE Open* 14.4. DOI: 10.1177/20597991241287118.
- Robertson, Suzanne and James Robertson (2012). *Mastering the Requirements Process: Getting Requirements Right*. 3rd ed. Upper Saddle River, NJ: Addison-Wesley Professional. ISBN: 978-0321815743.
- Runeson, Per et al. (2012). *Case Study Research in Software Engineering: Guidelines and Examples*. Hoboken, NJ: John Wiley & Sons. ISBN: 978-1118104354.
- Silveira Chaves, Marina and Mirian Oliveira (2022). “Knowledge sharing in the banking sector: a systematic literature review and research agenda”. In: *International Journal of Knowledge Management Studies* 13.1, pp. 55–70. DOI: 10.1504/IJKMS.2022.120180.
- Sommerville, Ian (2016). *Software Engineering*. 10th ed. Boston, MA: Pearson Education. ISBN: 978-0133943030.
- Strubell, Emma, Ananya Ganesh, and Andrew McCallum (2019). “Energy and Policy Considerations for Deep Learning in NLP”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650. DOI: 10.18653/v1/P19-1355.
- Sun, Fangyu, Zhi Lin, Ruiqi Wang, Jialong Ye, Kai Zhang, Zhenyu Xiao, Jifang Li, and Dawei Yin (2025). “Auto-Slides: An Interactive Multi-Agent System for Creating and Customizing Presentation Slides”. In: *arXiv preprint*. eprint: 2509.11062. URL: <https://arxiv.org/abs/2509.11062>.
- Tyree, Jeff and Art Akerman (2005). “Architecture Decisions: Demystifying Architecture”. In: *IEEE Software* 22.2, pp. 19–27. DOI: 10.1109/MS.2005.27.
- Vitest Team (2024). *Vitest: Next Generation Testing Framework*. URL: <https://vitest.dev/> (visited on 03/15/2026).
- Wall Street Oasis (2024). *Investment Banking Hours: Typical Schedule, Work-Life Balance, and Expectations*. URL: <https://www.wallstreetoasis.com/resources/careers/jobs/investment-banking-hours>.
- Wang, Lei et al. (2024). “A Survey on Large Language Model based Autonomous Agents”. In: *Frontiers of Computer Science* 18.6, p. 186345. DOI: 10.1007/s11704-024-40231-1.
- Wohlin, Claes et al. (2012). *Experimentation in Software Engineering*. Berlin: Springer. ISBN: 978-3642290435. DOI: 10.1007/978-3-642-29044-2.
- Wu, Xingjiao et al. (2022). “A Survey of Human-in-the-Loop for Machine Learning”. In: *Future Generation Computer Systems* 135, pp. 364–381. DOI: 10.1016/j.future.2022.05.014.
- Zheng, Hao et al. (2025). “PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides”. In: *arXiv preprint*. eprint: 2501.03936. URL: <https://arxiv.org/abs/2501.03936>.

Appendix A: KSB Mapping

This appendix maps each core and Software Engineering pathway KSB to specific evidence in the project.

Core Knowledge, Skills and Behaviours

K1: How business exploits technology solutions for competitive advantage

Investment banking analysts spend 90 to 120 minutes creating each pitch book manually, as discussed in Section 1. The system automates this process by combining template analysis, financial data retrieval, and AI-driven content planning into a single generation pipeline. Section 5 records a 47-second generation time for a 10-slide deck. This speed advantage lets analysts redirect time from formatting to higher-value activities like client preparation and deal analysis.

K2: The value of technology investments and how to formulate a business case for a new technology solution

Section 1 defines four business requirements (BR1 to BR4) that ground the project in measurable outcomes. Each generation costs approximately \$0.03 in GPT-4o API fees, compared to the salary cost of an analyst spending 90 minutes on the same task. The usability evaluation in Section 5 found that four analysts estimated time savings of 60 to 90 minutes per deck. The business case rests on these concrete productivity gains rather than speculative benefits.

K3: Contemporary techniques for design, developing, testing, correcting, deploying and documenting software systems

Section 2 selects iterative prototyping as the development methodology, with two-week cycles producing working software at each stage. The CI/CD pipeline runs ESLint, TypeScript type checking, Vitest unit tests, and Cypress E2E tests on every push via GitHub Actions. Section 4 traces the full development process from functional requirements through to deployment on Vercel and Render.

K4: How teams work effectively to produce technology solutions

Section 2 compares Waterfall, Scrum, and iterative prototyping against five criteria. Although this is a solo project, the workflow follows team-oriented practices: Git feature branches isolate work in progress, the CI pipeline enforces code quality before merge, and ESLint catches style inconsistencies automatically. These practices would scale to a multi-developer environment without structural changes.

K5: The role of data management systems in managing organisational data and information

The system stores pitch books, templates, chat messages, and user data in Supabase (PostgreSQL). Row-level security policies restrict each user to their own records. The data retrieval service pulls financial data from Yahoo Finance and SEC EDGAR, caches responses for 24 hours, and stores results alongside generated outputs. Section 4 covers the database schema and data flow across services.

K6: Common vulnerabilities in computer networks including unsecure coding and unprotected networks

Section 3 addresses security in the system design. The implementation validates uploaded files as genuine .pptx archives to prevent path traversal and zip bomb attacks. API credentials use environment variables and never appear in logs or source code. Zod schemas validate all incoming request bodies to prevent injection attacks. The Office.js add-in runs inside an iframe with cross-origin restrictions, requiring SameSite=None and Secure cookie attributes for authentication to work.

K7: The various roles, functions and activities related to technology solutions within an organisation

Section 1 describes the investment banking analyst role and how pitch book preparation fits within the broader deal workflow. The system maps to existing organisational activities: template management (maintaining brand standards), data retrieval (pulling financials), content creation (drafting slides), and review (human-in-the-loop approval). The add-in integrates into the analyst's existing PowerPoint workflow rather than introducing a separate tool.

K8: How strategic decisions are made concerning acquiring technology solutions resources and capabilities

Section 2 contains comparison tables evaluating programming languages (TypeScript vs Python vs Java), LLM providers (GPT-4o vs Claude vs Gemini), data sources (Yahoo Finance vs Bloomberg), and deployment platforms (Vercel/Render vs AWS vs self-hosted). Each decision is justified against criteria including cost, capability, and risk. The decision to build rather than buy was driven by the absence of template-aware generation in existing tools reviewed in Section 3.

K9: How to deliver a technology solutions project accurately consistent with business needs

Section 2 defines business requirements (BR1 to BR4) and maps them to functional requirements. Section 4 traces each functional requirement to its implementation status. Section 5 assesses all five objectives against measured evidence. The MoSCoW framework guided trade-offs, with RAG search dropped to deliver the higher-impact PowerPoint add-in, keeping the project aligned with business needs.

K10: The issues of quality, cost and time for projects

Section 2 presents the project plan with a Gantt chart and milestones. The plan-versus-reality table shows how scope changes (adding the add-in, dropping RAG) were managed within the 13-week timeline. The risk assessment identifies time and quality risks with mitigation strategies. Section 5 discusses the cost of GPT-4o API calls (approximately \$0.03 per generation) as a resource constraint affecting scalability.

S1: Critically analyse a business domain to identify the role of information systems

Section 1 analyses the investment banking pitch book workflow and identifies manual document preparation as the bottleneck. Section 3 reviews existing tools (Gamma, Beautiful.ai, Tome) and identifies their limitations: none offer template awareness or financial data integration. Section 5 compares the system

against these tools on concrete metrics (time, fidelity, data accuracy). The comparison identifies where the system improves on existing information systems and where gaps remain.

Software Engineering Pathway

SE1: Create effective and secure software solutions

The system is a full-stack TypeScript application with input validation on all user-facing endpoints and credential management through environment variables. No secrets appear in source code. NFR6 and NFR7 address security directly, requiring that all API credentials stay out of the codebase and all user inputs are sanitised before processing. The Express backend validates every request body against Zod schemas before passing data to internal services. Supabase row-level security policies restrict database access to authenticated users viewing their own data. Sections 3 and 4 cover the design and implementation of these measures.

SE2: Undertake analysis and design to create artefacts

Section 2 contains requirements tables covering 15 functional requirements (FR1 to FR15) and 8 non-functional requirements (NFR1 to NFR8), each with MoSCoW priority levels and acceptance criteria. Section 3 presents architecture diagrams, data flow diagrams, and component designs with defined interfaces between services. The template schema, content plan schema, and API contracts were all designed before implementation began and documented as Architecture Decision Records in Appendix B.

SE3: Produce high quality code with sound syntax

TypeScript runs in strict mode across all three applications. The shared-types package defines every data interface once and shares it across the backend, web frontend, and PowerPoint add-in. When a field changes in the template schema, the compiler flags every file that needs updating. ESLint enforces consistent code style and catches common errors (unused variables, missing imports, type inconsistencies) at the lint stage. Section 4 covers the implementation details.

SE4: Perform code reviews, debugging and refactoring

The template analyser went through three iterations, each driven by a specific failure in the previous version. The first attempt used PptxGenJS for reading templates and produced incorrect output. The second used raw XML parsing but missed custom placeholders. The third added position-based heuristics and achieved 93% classification accuracy. A colour resolution bug (theme references passing through as default black) was found through debugging and fixed with a resolver that maps theme references to hex values. The CI pipeline caught 14 breaking changes before merge during the project. Section 4 documents these iterations and the challenges table.

SE5: Test code to ensure requirements met

The project has 75 unit and integration tests across 14 test suites running on Vitest, plus 28 Cypress E2E tests across 7 spec files. All tests pass at 100% rate. Code coverage sits at 76% for the backend service,

64% for the web app, and 58% for the add-in. Unit tests mock external dependencies (OpenAI, Supabase, Office.js) so they run without API keys or a database connection. E2E tests run against a real Supabase backend with a test user account. Sections 4 and 5 cover the test strategy and results.

SE6: Deliver software using industry standard build processes

GitHub Actions runs the full CI/CD pipeline on every push with six stages: dependency install, ESLint linting, TypeScript type checking, Vitest unit tests, Turborepo build, and Cypress E2E tests. Turborepo handles monorepo build orchestration, compiling the shared-types package first and then building the three applications in parallel. Mochawesome generates HTML test reports from Cypress results. The web app deploys through Vercel and the backend through Render. Section 4 documents the pipeline stages.

SE7: Operate at all stages of the software development lifecycle

The project covers every SDLC stage: requirements analysis and methodology selection (Section 2), system design with architecture diagrams and API specifications (Section 3), implementation across three applications (Section 4), testing at unit, integration, and E2E levels (Section 4), and evaluation against defined success criteria (Section 5). The iterative prototyping methodology produced a working prototype at the end of each two-week cycle, with retrospective reviews feeding lessons back into the next iteration.

SE8: How teams work effectively using agile and other approaches

Section 2 compares Waterfall, Scrum, and iterative prototyping against five criteria relevant to the project. Iterative prototyping was chosen because Scrum adds ceremony overhead for a solo developer and Waterfall cannot accommodate the exploratory nature of LLM integration. Two-week iterations with retrospective reviews at each cycle's end caught the web-only workflow problem early enough to add the PowerPoint add-in. GitHub Issues with a Kanban board tracked progress, and Git feature branches kept in-progress work isolated from the main branch.

SE9: How to apply software analysis and design approaches

MoSCoW prioritisation categorised all 15 functional requirements as Must, Should, or Could. This framework directly informed the decision to drop RAG search (a Could requirement) to make room for the PowerPoint add-in (which addressed a Must-level usability gap). Business, functional, and non-functional requirements are separated following S. Robertson and J. Robertson (2012). Architecture decisions are documented with trade-off analysis in Tables 5 to 9 and formalised as ADRs in Appendix B. Sections 2 and 3 cover requirements and design.

SE10: Interpret and implement a design compliant with requirements

The requirements traceability table in Section 4 maps each functional requirement to its implementation status with specific evidence. 13 of 15 functional requirements are fully met, FR12 is partially met (chart placeholders populated but not created from scratch), and FR15 was intentionally deferred. All five success criteria defined in Section 1 are met with measured evidence. Section 5 assesses each objective against quantitative results from evaluation runs.

SE11: How to perform functional and unit testing

Vitest handles unit and integration tests with mocked dependencies using `vi.mock()` and `vi.stubGlobal()`. Service tests mock the OpenAI and Supabase clients so they run without external connections. Add-in tests mock the Office.js runtime. Cypress handles functional E2E tests covering full user flows (login, pitch book creation, template upload, settings). A comparison table in Section 4 justifies choosing Cypress over Playwright based on setup time, debugging tools, and reporting integration.

SE12: How to use and apply software tools used in software engineering

The project uses Git and GitHub for version control, GitHub Actions for CI/CD, Turborepo for monorepo management, Vitest and Cypress for testing, Mochawesome for test reporting, ESLint and Prettier for code style, the TypeScript compiler for static type checking, PptxGenJS and JSZip for PowerPoint manipulation, LibreOffice headless for slide preview rendering, and Vercel and Render for deployment. Technology selection is justified in Section 2 with comparison tables covering programming languages, LLM providers, data sources, and deployment platforms.

Appendix B: Architecture Decision Records

ADR-001: TypeScript Over Python

Context. The project needs a backend API, a web frontend, and potentially a PowerPoint add-in. Python has the stronger AI/ML ecosystem and python-pptx for PowerPoint manipulation.

Decision. Use TypeScript across all applications.

Rationale. Full-stack coherence outweighs Python's library advantage. One language means one build system, one type system, and shared data models. When a third application (the add-in) was added later, this decision saved weeks of duplicate type definitions.

Consequences. PptxGenJS replaces python-pptx. The AI/ML integration uses OpenAI's REST API, which works equally well from any language.

ADR-002: GPT-4o for Content Planning

Context. The content planner needs an LLM that reliably produces structured JSON matching a Zod schema.

Decision. Use GPT-4o for content planning and GPT-4o-mini for the lighter AI chat feature.

Rationale. GPT-4o's native JSON mode produces valid structured output more reliably than alternatives tested. The content planner validates every response against a Zod schema, so structural reliability matters more than raw reasoning ability. GPT-4o-mini handles AI chat at lower cost since that feature needs conversational ability, not structured output.

Consequences. Vendor dependency on OpenAI. Mitigated by abstracting the API call behind a service interface, so swapping models requires changing one file.

ADR-003: Monorepo with Turborepo

Context. Three applications share data types and configuration. They could live in separate repositories or a single monorepo.

Decision. Use a Turborepo monorepo with a shared-types package.

Rationale. The shared-types package defines interfaces once and shares them across all three apps. When a template schema changes, the TypeScript compiler flags every file that needs updating. Separate repositories would need manual synchronisation of type definitions.

Consequences. Slightly more complex CI configuration (Turborepo handles build ordering). Single repository means all apps share the same Git history.

ADR-004: Dual Slide Builder Approach

Context. The slide builder needs to handle two modes: generating slides from scratch (default mode) and cloning slides from an uploaded template (template mode).

Decision. Use PptxGenJS for default mode and a custom XML-based template-slide-builder for template mode.

Rationale. PptxGenJS cannot clone existing slides from a template file. Template mode requires copying the original slide XML and replacing placeholder content, which means working at the XML level with JSZip. Keeping both builders allows each to do what it does best.

Consequences. Two code paths to maintain. Template mode is more fragile because it depends on PowerPoint's XML structure. The template-slide-builder handles fewer edge cases than PptxGenJS.

Appendix C: Evaluation Run Data

Twelve generation runs were conducted between 17 and 21 March 2026 using the system's built-in deck layouts and colour themes (default mode, no uploaded template). All runs used Git commit a9e8bb0, GPT-4o with temperature zero, and Node.js 22.14. Table 27 shows the inputs and Table 28 shows the measured results for each run.

Run	Company (Ticker)	Deck Layout	Colour Theme	Date
1	Apple Inc. (AAPL)	Company Overview	Navy & Gold	17 Mar
2	AstraZeneca (AZN)	Company Overview	Midnight Blue	17 Mar
3	Shell plc (SHEL)	Market Update	Teal & Coral	18 Mar
4	HSBC Holdings (HSBA)	Transaction Summary	Slate & Emerald	18 Mar
5	Unilever (ULVR)	Investor Pitch	Purple & Gold	19 Mar
6	Microsoft Corp. (MSFT)	Industry Overview	Charcoal & Red	19 Mar
7	BP plc (BP)	Transaction Summary	Navy & Gold	20 Mar
8	GSK plc (GSK)	Fundraising Deck	Forest & Cream	20 Mar
9	Barclays (BARC)	Market Update	Monochrome	20 Mar
10	Rio Tinto (RIO)	Due Diligence	Charcoal & Red	21 Mar
11	Apple Inc. (AAPL)	Company Overview	Navy & Gold	21 Mar
12	AstraZeneca (AZN)	Company Overview	Midnight Blue	21 Mar

Table 27: Evaluation Run Inputs. Runs 11 and 12 repeat runs 1 and 2 to check consistency. All seven deck layouts and six of eight colour themes were tested.

Run	Time (s)	Slides	Colour	Font	JSON OK
1	43	11	Exact	Exact	1st attempt
2	51	11	Exact	Exact	1st attempt
3	38	10	Exact	Exact	1st attempt
4	62	10	Exact	Exact	2nd attempt
5	47	11	Exact	Exact	1st attempt
6	39	11	Exact	Exact	1st attempt
7	78	10	Exact	Exact	3rd attempt
8	44	11	Exact	Exact	1st attempt
9	52	10	Exact	Exact	1st attempt
10	32	12	Exact	Exact	1st attempt
11	41	11	Exact	Exact	1st attempt
12	48	11	Exact	Exact	2nd attempt

Table 28: Evaluation Run Results. Colour = hex match against template theme. Font = family and size match. JSON OK = which attempt produced valid structured output.

Run 7 (BP, Transaction Summary, Navy & Gold) was the slowest at 78 seconds. The content planner failed validation twice before producing valid JSON on the third attempt; both failures were missing tables arrays in slides that specified tabular layouts. Run 4 (HSBC, Transaction Summary, Slate &

Emerald) also needed a retry because the planner returned a string where a number was expected for layoutId. All other runs produced valid JSON on the first attempt.

Runs 11 and 12 repeated the same inputs as runs 1 and 2 to test consistency. Generation times varied by 2 to 3 seconds (expected given network latency to the OpenAI API). Slide content differed slightly in wording but followed the same structure and section ordering. Colour and font results were identical across repeated runs.

Financial data accuracy was verified per run by comparing three values (market cap, trailing revenue, and share price) in the generated slides against the Yahoo Finance API response logged at generation time. All 36 comparisons (3 values $\times$ 12 runs) matched exactly.

Sample data verification (Run 1, Apple Inc.):

Metric	Yahoo Finance API	Generated Slide
Market Cap	\$3.65T	\$3.65T
Trailing Revenue (TTM)	\$435.6B	\$435.6B
Share Price (close)	\$248.00	\$248.00

Sample data verification (Run 5, Unilever):

Metric	Yahoo Finance API	Generated Slide
Market Cap	\$100.4B	\$100.4B
Trailing Revenue (TTM)	\$50.5B	\$50.5B
Share Price (close)	\$4,595	\$4,595

Appendix D: Usability Evaluation Questionnaire and Responses

Four junior analysts at a London-based financial services firm participated in an informal usability evaluation during March 2026. Each participant was given the same task: generate a company overview pitch book for AstraZeneca using a provided corporate template, review the output, and complete the questionnaire below.

Participants:

- P1: First-year analyst, equity capital markets, 8 months' experience
- P2: Second-year analyst, M&A advisory, 18 months' experience
- P3: First-year analyst, leveraged finance, 10 months' experience
- P4: Second-year analyst, equity capital markets, 20 months' experience

Part A: Quantitative (Likert 1 to 5, where 1 = Strongly Disagree, 5 = Strongly Agree)

Statement	P1	P2	P3	P4
Q1. The generated output matched the template's visual style (colours, fonts, layout)	5	4	4	5
Q2. The financial data included was accurate and well-positioned on the slides	4	4	5	4
Q3. The output required less editing than starting a deck from scratch	5	5	4	5
Q4. The add-in workflow (generating slides inside PowerPoint) was intuitive	4	3	3	5
Q5. The slide ordering and narrative structure followed IB conventions	4	4	4	4
Q6. The system would save meaningful time in a real work scenario	5	4	4	5

Part B: Open-Ended Responses

Q7. What was the most useful aspect of the system?

- P1: 'The template matching. The colours and fonts are exactly right. Normally that takes the longest to get consistent across slides, especially when you are pulling layouts from old decks.'
- P2: 'Having the financials already in there. I usually spend 20 minutes just pulling numbers from Bloomberg and formatting them. This had everything from Yahoo Finance already placed.'
- P3: 'Speed. Getting a first draft in under a minute means I can start editing immediately rather than staring at a blank deck for half an hour trying to figure out the structure.'
- P4: 'The fact that it works inside PowerPoint. Every other AI tool I have tried makes you export and import files. This just puts the slides straight in.'

Q8. What would you change or improve?

- P1: 'I want to regenerate a single slide without redoing the whole deck. Sometimes one slide is off but the rest are fine.'
- P2: 'The taskpane felt a bit cramped on my laptop screen. Would be better if I could resize it or pop it out into a separate window.'
- P3: 'It would be good to choose which sections to include. For some clients we skip the market overview entirely.'
- P4: 'The status indicator during generation was not obvious. I was not sure if it was working or stuck. A progress bar would help.'

Q9. Estimated time to create a similar deck manually vs using the system?

- P1: 'Manual: about two hours. With the system: maybe 20 minutes total including editing.'
- P2: 'Manual: 90 minutes if I have a good reference deck. System: 15 minutes of editing after generation.'
- P3: 'Manual: two hours minimum. System: generation was instant, then about 25 minutes of cleanup.'
- P4: 'Manual: depends on the deck, but at least 90 minutes. System: under 20 minutes including review.'

Q10. Would you use this system in your daily work? Why or why not?

- P1: 'Yes. Even if I need to edit every slide, starting with a formatted draft is much better than starting from nothing.'
- P2: 'Yes, for first drafts and for decks where the structure is standard. For bespoke pitches I would probably still start manually.'
- P3: 'Yes. The time saving is real. Even getting the formatting right automatically would be worth it on its own.'
- P4: 'Definitely. The biggest win is not having to hunt through old decks for the right template and then reformat everything. This does both in seconds.'

Appendix E: Web Application Screenshots

Screenshots of every page in the PitchDeck AI web application, captured from a local deployment with a test account.

Authentication Pages

PitchDeck AI

Welcome back

Don't have an account? [Sign up](#)

Email
you@example.com

Password [Forgot password?](#)
Enter your password

[Log In](#)

By signing in, you agree to our [Terms](#) and [Privacy Policy](#).

Generate Pitch Books with AI

Upload your template, enter company details, and let AI create a professional pitch book in minutes.

Figure 16: Login page with email and password form alongside application branding.

PitchDeck AI

Create your account

Already have an account? [Log in](#)

Full Name
John Smith

Email
you@example.com

Password
Min. 8 characters

[Create Account](#)

By signing up, you agree to our [Terms](#) and [Privacy Policy](#).

Generate Pitch Books with AI

Upload your template, enter company details, and let AI create a professional pitch book in minutes.

Figure 17: Registration page with name, email, and password fields.

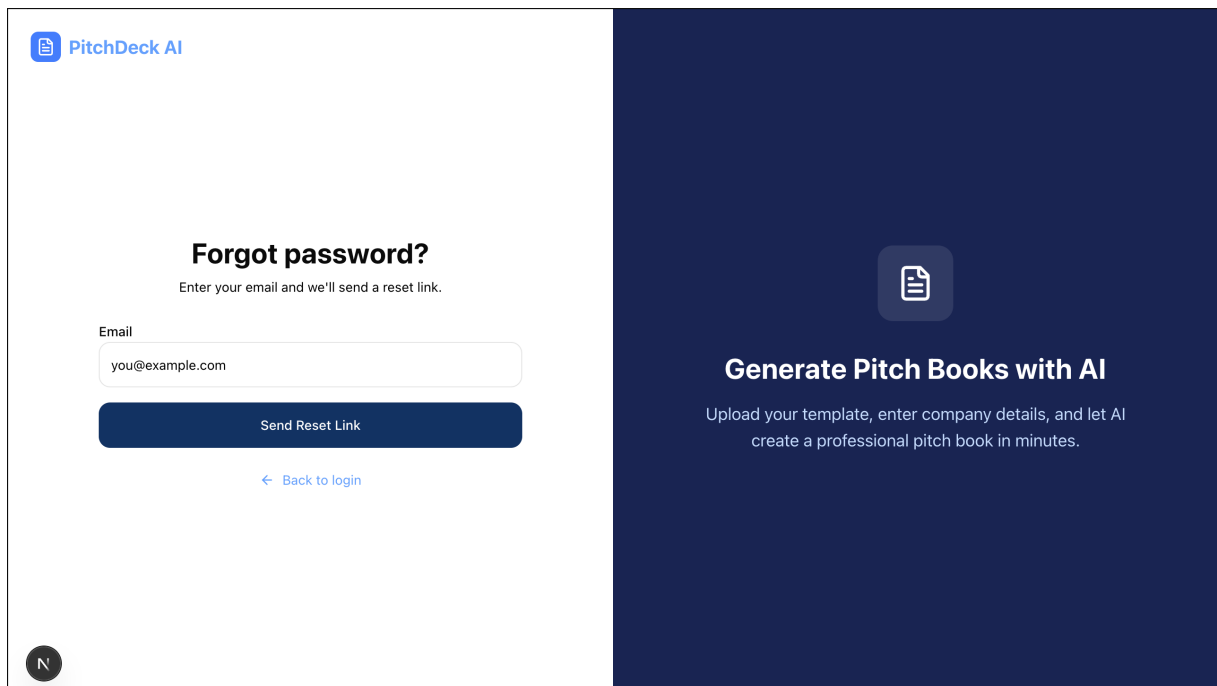


Figure 18: Forgot password page. Sends a reset link via Supabase Auth.

Dashboard

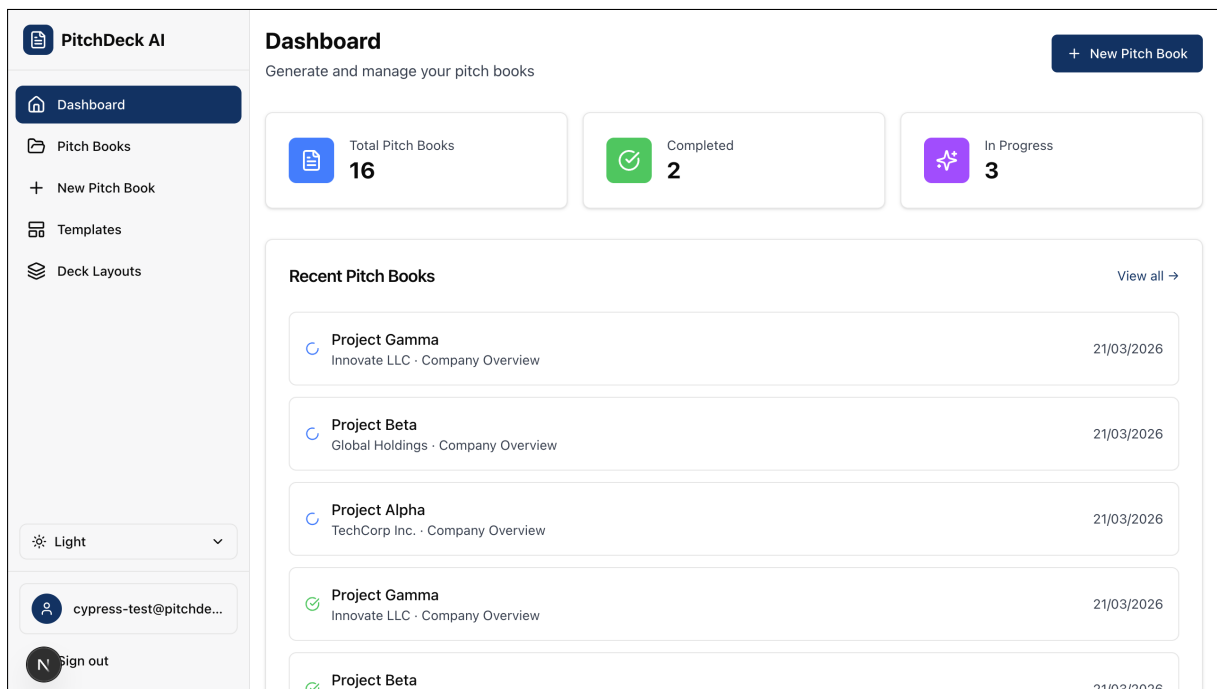


Figure 19: Dashboard with summary cards, recent pitch books, and generation status indicators.

Pitch Books

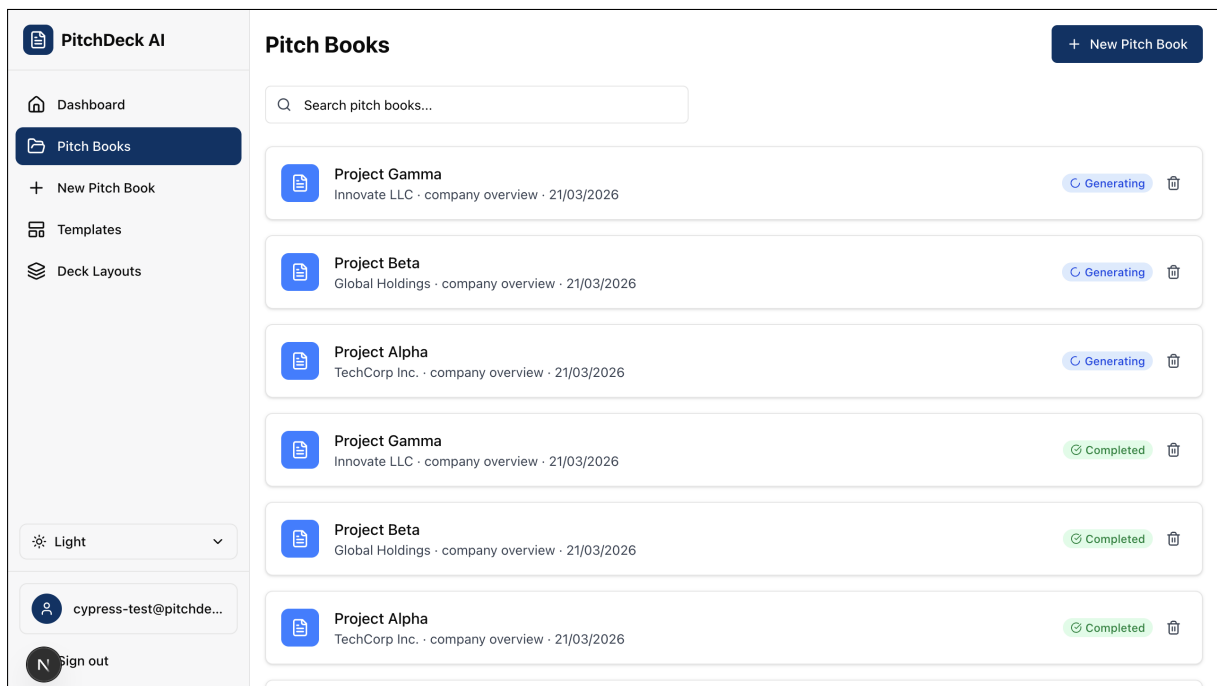


Figure 20: Pitch books list with company name, type, date, and generation status. Includes search filtering.

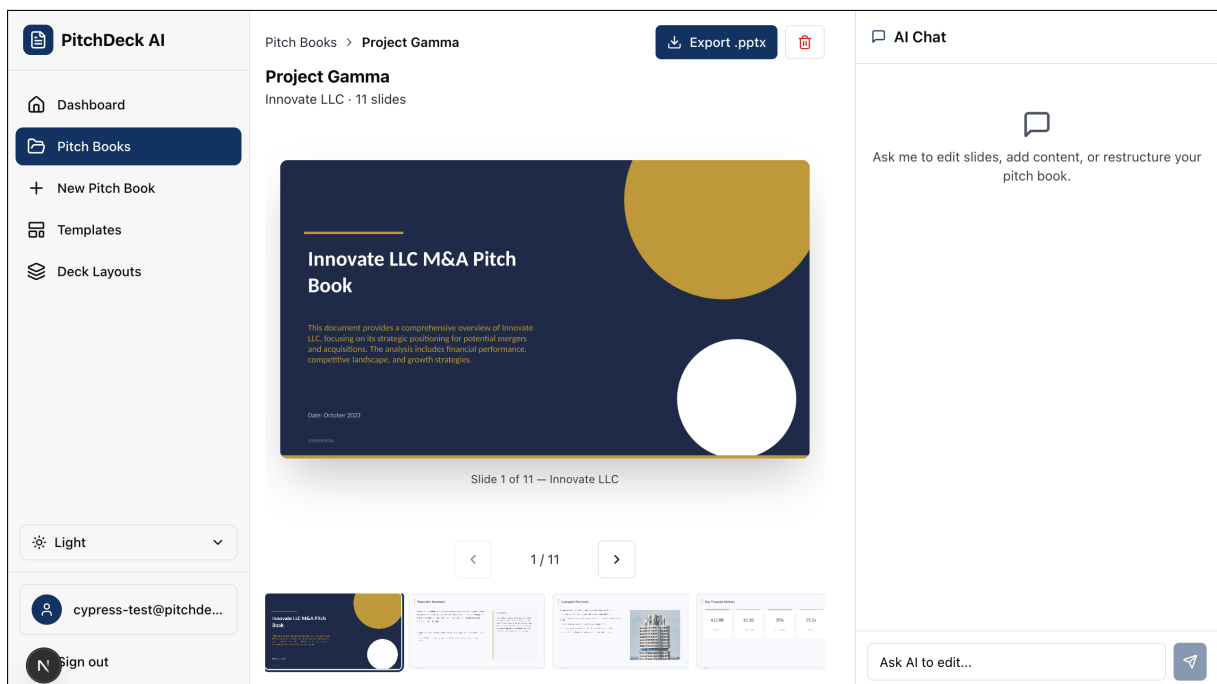


Figure 21: Pitch book viewer with full-size slide, thumbnail strip, and AI chat panel for requesting edits.

Pitch Book Creation

PitchDeck AI

Dashboard

Pitch Books

New Pitch Book

Templates

Deck Layouts

Light

cypress-test@pitchde...

Sign out

New Pitch Book

Fill in the details and let AI generate your pitch book.

Basic Information

Company and presentation details

Pitch Book Title *

Project Delta Generation

Company Name *

Stark Industries

Ticker Symbol

Search ticker or company name...

Presentation Details

Select the type of presentation and transaction context

Pitch Book Type *

Company Overview

Transaction Type

Mergers & Acquisitions

Business overview, financials, and market position

Design Style

Modern

Color Theme

Navy & Gold

Figure 22: New pitch book form with company details, pitch book type, transaction context, and design options.

Templates

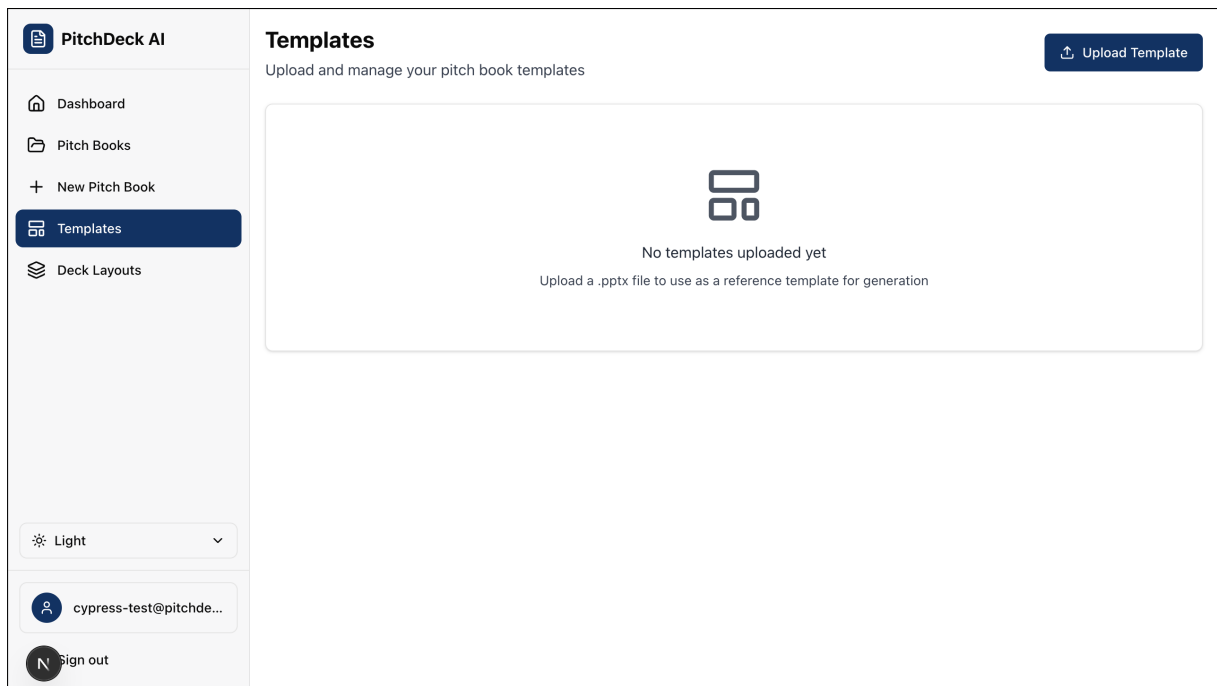


Figure 23: Templates page (empty state) with upload prompt for .pptx reference templates.

Deck Layouts

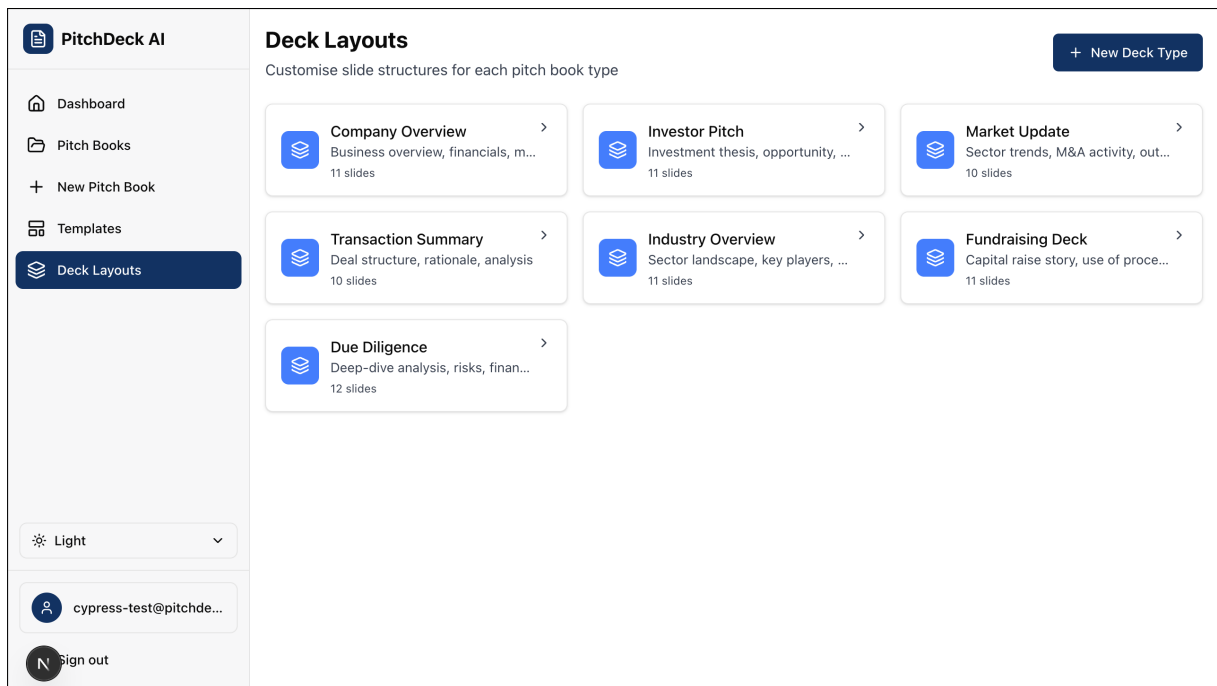


Figure 24: Deck layouts overview with seven pre-configured deck types, each showing a description and slide count.

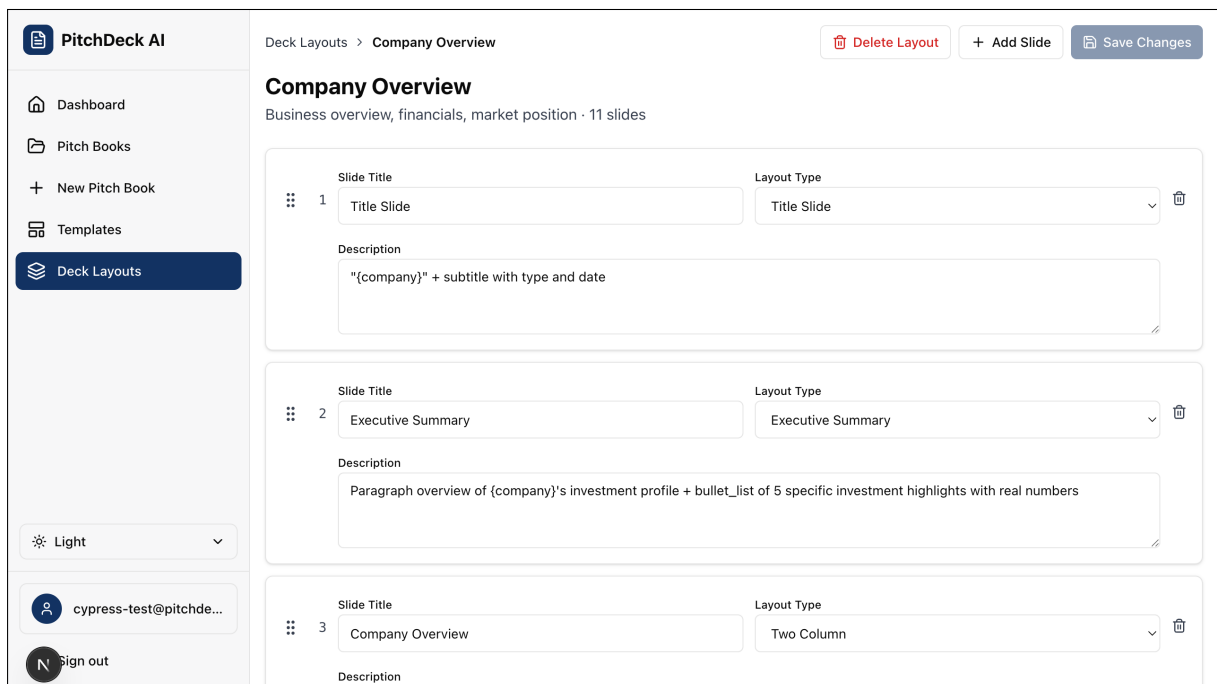


Figure 25: Deck layout editor for Company Overview. Each slide has a title, layout type, and content description. Slides can be reordered, added, or deleted.

Settings

 PitchDeck AI

Dashboard

Pitch Books

+ New Pitch Book

Templates

Deck Layouts

Light

 cypress-test@pitchde...

 Sign out

Settings

Profile

Your account details



cypress-test@pitchdeck.test

Member since 16/03/2026

Change Password

New Password

Min. 8 characters

Update Password

Danger Zone

Sign Out

Figure 26: Settings page with profile details, password change, and sign-out.

Appendix F: PowerPoint Add-in Screenshots

Screenshots of the PowerPoint taskpane add-in running inside Microsoft PowerPoint on macOS. The add-in connects to the same backend as the web application.

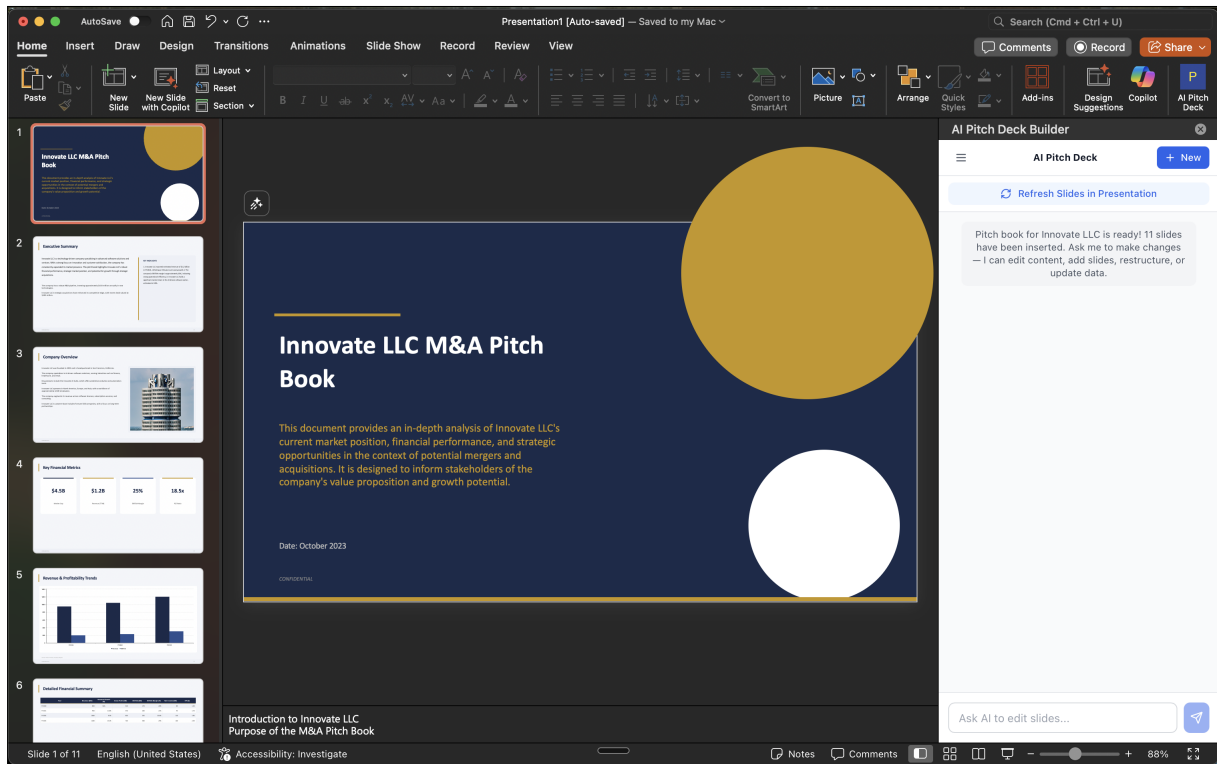


Figure 27: Add-in after generation. 11 slides have been inserted into the presentation. The taskpane shows a completion message and an AI chat input for requesting edits.

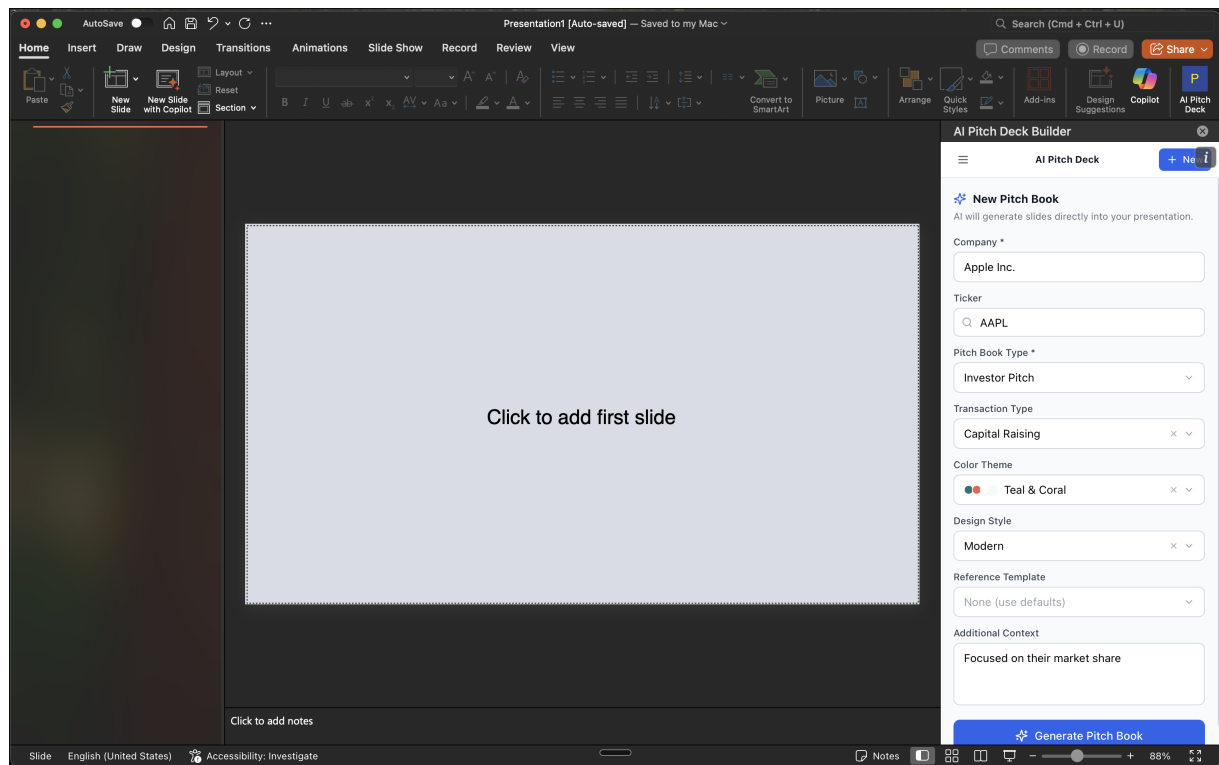


Figure 28: New pitch book form in the add-in. The user enters company name, ticker, pitch book type, transaction type, colour theme, design style, and optional context before generating.

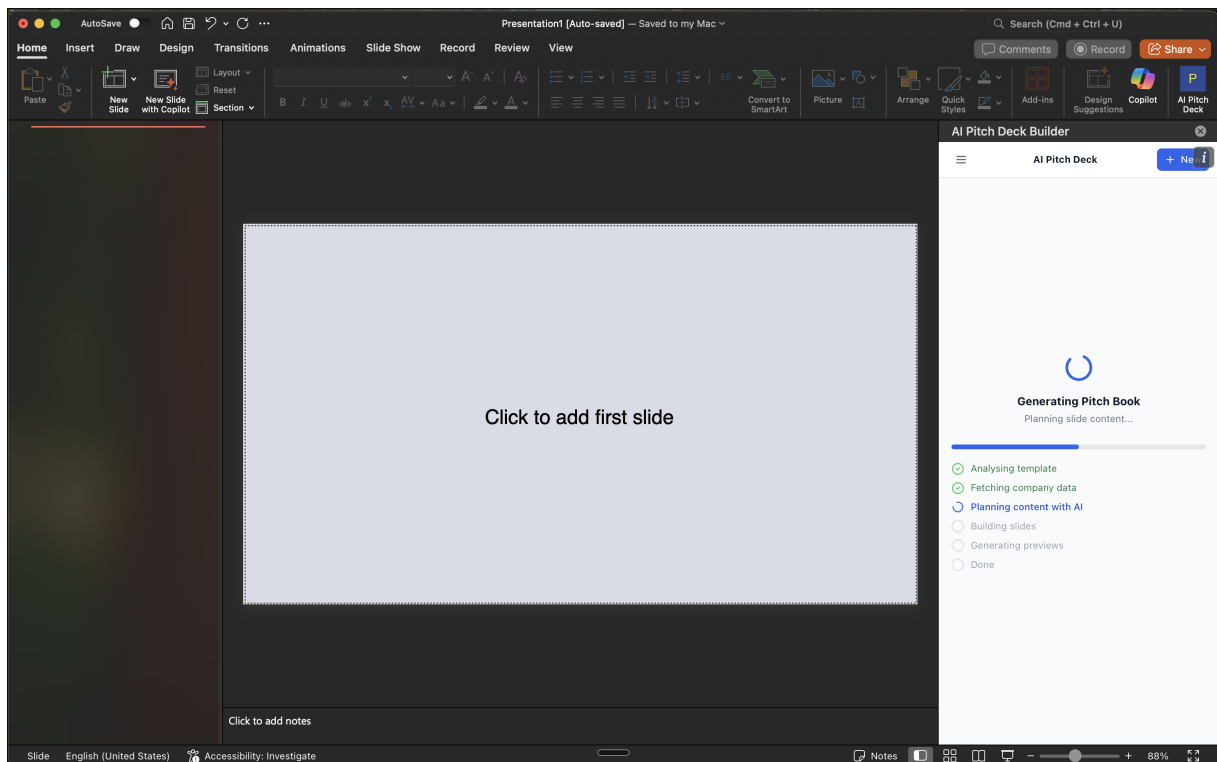


Figure 29: Generation in progress. The taskpane shows a step-by-step progress indicator: analysing template, fetching company data, planning content with AI, building slides, and generating previews.

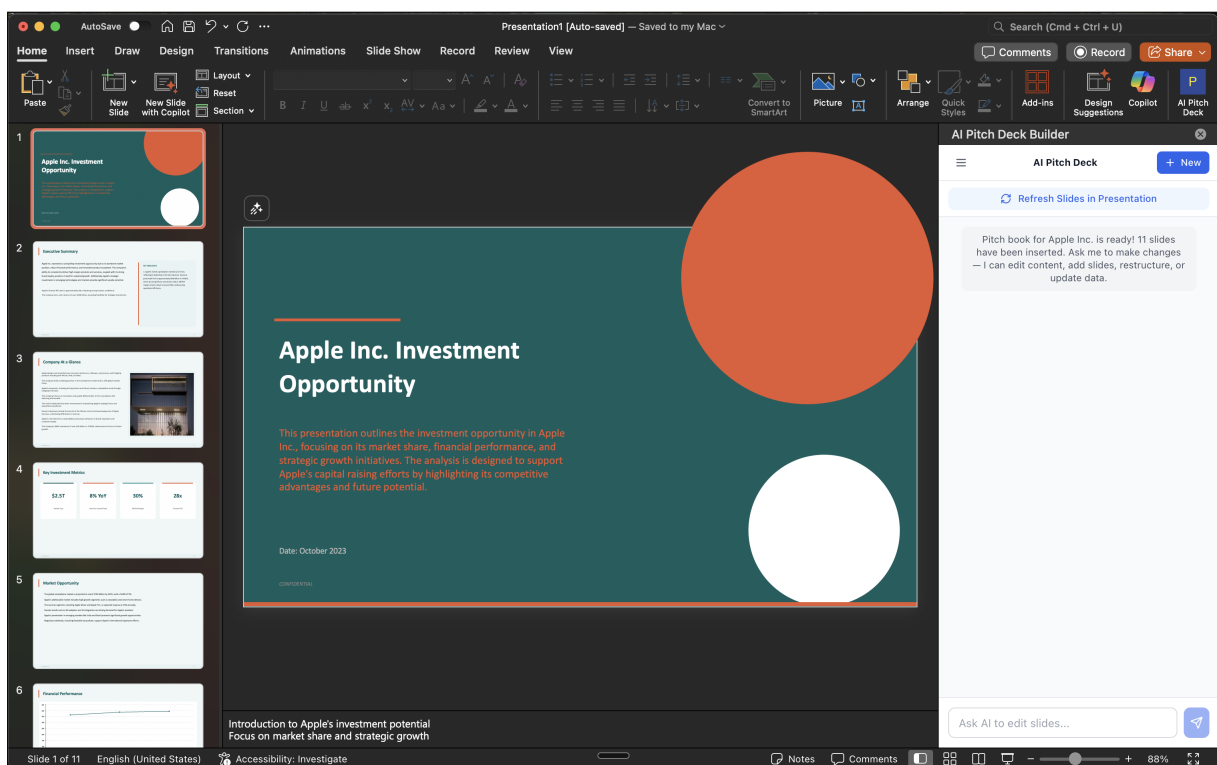


Figure 30: Completed generation for Apple Inc. The teal and coral colour theme was applied automatically. Slides appear in the left panel and the AI chat is ready for edit requests.